Predicting Alzheimer's Disease Using Statistical Learning Algorithms

Francisco Canova
College of Engineering & Computer Science
University of Central Florida
Orlando, Florida
fr396029@ucf.edu

Cameron Cummins
Department of Statistics and Data Science,
College of Sciences
University of Central Florida
Orlando, Florida
ca746665@ucf.edu

Spencer Leighty
Department of Statistics and Data Science,
College of Sciences
University of Central Florida
Orlando, Florida
sp786493@ucf.edu

***Abstract*—This report outlines the development of a predictive model for Alzheimer's disease based on lifestyle, demographic, medical, and genetic risk factors. It details a thorough exploration of the data, and the testing of six different types of models.**

Keywords—Alzheimer's, diagnosis, statistical learning, modelling

I. Executive Summary

In performing this statistical analysis, we are looking to build a model that can predict whether someone has Alzheimer's disease based on factors such as genetics, diet, BMI, country, and more. Early detection is critical, as it can allow for preventive action and reduce the cost and intensity of late-stage care. We successfully developed a bagging ensemble model that correctly classifies 72.39% of observations as positive or negative for Alzheimer's. The model was validated using a 75/25 train-test split, demonstrating reasonable performance for a real-world screening tool.

Patients can use this to see whether they should be recommended to go through the expensive and thorough testing process based on their risk factors. For example, an online quiz on a health plan's website could determine, using this model, whether a patient should seek medical testing, and if so, recommend them to book an appointment. A health insurance or other health company can also use the model to determine at what age someone is predicted to have Alzheimer's (i.e. continue to increase the age until the prediction flips from 0 to 1, thereby determining the age at which the person could develop the disease holding all other factors constant). This could be extremely helpful in not only developing their audience but also motivating patients to change their habits. In other words, if a patient has an age at which they are statistically likely to develop Alzheimer's based on their diet, physical activity level, etc., they may be more motivated to become a healthier version of themselves.

Overall, the most important predictors of Alzheimer's Disease on their own were: Genetic Risk Factor, Family History of Alzheimer's, and Age. These variables were all highly correlated to a positive result. Other variables, like diet, physical activity level, and sleep quality, work together to paint an even bigger picture of Alzhiemer's.

While the model shows promise, it is not a substitute for clinical diagnosis. There remains a risk of false positives and false negatives. Therefore, the model should be used as a supplementary tool rather than a standalone decision-maker. Ethical considerations, including data privacy, the psychological impact of prediction, and equity across demographic groups, must be addressed before deployment. Future improvements could include exploring additional features such as cognitive test scores or biomarkers, applying more robust cross-validation techniques, and calibrating the model for use across diverse populations. With refinement, this model could play a valuable role in proactive healthcare and disease prevention.

II. Introduction

Our project aims to build a predictive model for Alzheimer's disease using a dataset sourced from Kaggle, which was collected between February 6th, 2023, and June 21st, 2024. The dataset includes a total of 74,283 observations with 25 variables - 20 categorical and 4 numeric - and reflects real-world demographic and regional biases and risk factors.

III. Problem Definition

We defined our problem as whether we can predict if someone has Alzheimer's Disease based on a variety of risk factors. We are more interested in accuracy, rather than interpretability, so our choices for model selection will reflect this in later steps.

IV. Data Collection

A. Summary of Data

The dataset, collected on Kaggle, describes risk factors for Alzheimer's disease, including demographic, lifestyle, medical, and genetic variables, for 74,283 observations. It also includes whether the person in question has Alzheimer's or not. The dataset has a biased distribution which reflects real-world discrepancies across regions and the data itself was collected from February 6th, 2023, to June 21st, 2024. The dataset has 25 variables, with 20 predictors being categorical and the other 4 being numerical. The data can be accessed on Kagle:

https://www.kaggle.com/datasets/ankushpanday1/alzheimers-prediction-dataset-global/data

The following is a list of all the different independent variables within the dataset: Country, Age, Gender, Education Level, BMI, Physical Activity Level, Smoking Status, Alcohol Consumption, Diabetes, Hypertension, Cholesterol Level, Family History of Alzheimer's, Cognitive Test Score, Depression Level, Sleep Quality, Dietary Habits, Air Pollution Exposure, Employment Status, Marital Status, Genetic Risk Factor(whether or not the person has the APOE-ε4 allele), Social Engagement Level, Income Level, Stress Levels, Urban vs. Rural Living. While the target variable in the data set is Alzheimer's Diagnosis, which is whether or not the individual is diagnosed with Alzheimer's.

B. Train and Test Split

The train test split we used was 75/25 across all the models. We also stratified the two datasets so that we had an equal distribution of both classes across the training and testing dataset, with 41% being diagnosed with Alzheimer's and 59% were not diagnosed with Alzheimer's.

V. Exploratory Data Analysys

A. Univariate Analysis

We analyzed the distribution of the four numeric predictors using histograms. They seem to loosely follow a uniform distribution. To try to transform the numeric predictors to follow a normal distribution, we tried the log transformation, square root transformation, and the inverse transformation. None of these seemed to correct the distribution and therefore we decided to move forward with the untransformed data. We also wanted to ensure there was no target leakage so that our later metrics can be reliable. So, we looked through the data and no variables appeared to be examples of target leakage. We analyzed the correlation matrix to make sure this was the case as seen in *Figure 1.15* where no factors have a high enough correlation to be of concern.

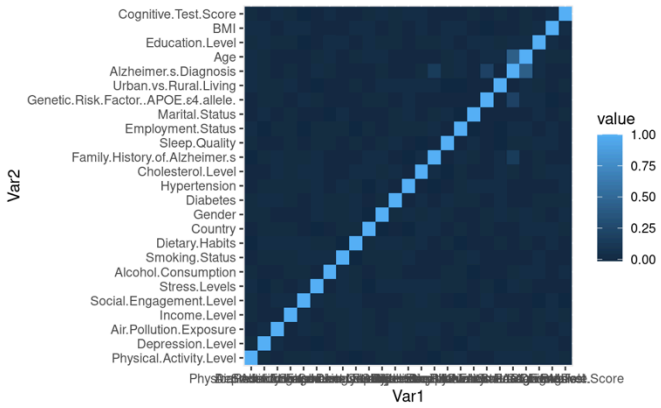

Figure 1.15 - The correlation matrix shows that there is no collinearity as all the variables that are correlated with the target variable.

B. Bivariate Relationships

For the numeric data, we compared the box plots of the data for the two levels of Alzheimer Diagnosis: Yes or No. To analyze whether the numeric predictor has a significant impact on Alzheimer Diagnosis, we compared the means and the spread of the plots. From the four predictors - Age, BMI, Education Level, and Cognitive Test Score - the only predictor that showed a significant change in mean between the levels was Age.

C. Dimensionality Reduction

Dimensionality reduction will be done in much further depth later in the study but as a preliminary we looked at the Country variable. Country is a categorical variable with 20 levels which would lead to a high-dimension model if all the dummy variables were included. To correct this, we grouped up the countries by continent and analyzed the means of each. However, the mean of each country was very similar when following this approach (i.e. they converged to around 0.4), and some continent levels included very little observations when compared to the others. The second approach we took was to group up like means - those that were above 0.42 (Russia, India, Brazil, Mexico, and South Africa) were grouped and those that were below 0.42 were grouped - into two levels. This served to reduce the dimensionality as well as maintain the behavior of each level.

D. Interaction Examination and Collinearity Concerns

To address interactions, we looked at some predictors that would seemingly see like they interact (e.g. BMI and Dietary Habits). From there we constructed the appropriate plot to

judge whether the power of one predictor depends on the level of another predictor. From our preliminary interaction examination, predictors that would seem like they would interact, were not. We chose to move on without interaction terms, however, we would keep in mind this strange detail in our final interpretation. To address collinearity concerns we constructed a correlation matrix shown in *Figure 1.15*. From the figure it is easy to tell that there is an exceptionally low correlation between the predictors and a strong correlation between the predictors and the response variable.

VI. Model Construction

A. Logistic & Probit Regression Models

We fitted a logistic model, both without an offset and one with age as an offset as we believed that it would be the most optimal feature to use as an offset. As seen in *Figure 2.1* the respective AICs are 60475.75, 68015.67, and 60405.24 for the logistic regression without an offset, the logistic regression with an offset, and the probit regression. From here we can see that the probit model has the lowest AIC score, so we selected that model to move forward with stepwise feature selection.

B. Stepwise Feature Selection

For stepwise feature selection with categorical variables, it is important to binarize the predictors with three or more levels so that it allows the feature selection function to view each level instead of having to choose between adding and removing all the dummy variables of the predictor entirely. Since our goal is to increase the predictive power of our model without overfitting, we did forward selection for AIC to remove insignificant predictors but not penalizing too heavily for more complex models. The predictors that survived the process were Age, Genetic Risk Factor for the APOE e4 allele, Family History of Alzheimer's, Country, Urban vs Rural Living, and Income Level Medium. The summary of the reduced probit model is given in *Figure 2.1*.

C. Principal Component Analysis

In addition to the Probit model alone, we performed a probit model with principal component analysis (PCA). In this technique, we attempted to reduce dimensionality by reducing the number of variables while keeping as much important information as possible. While this reduced our final model's dimensionality to only 4 variables, our model had a test accuracy of 70.72%, which was not as good as the probit model without PCA. The model also had a ROC AUC of 0.7438, not as high as the probit model's 0.7879. Thus, we decided not to move forward with this model.

D. Elastic Net Regression

On top of the more simplistic regression models above, we decided to fit an elastic net model on the same data. In order to select the optimal lambda we ran a cross validation process with different lambdas, and plotted them in different figures for the respective values of 0, 0.25, 0.5, 0.75, and 1. The best performing model, using testing accuracy as the metric, had an alpha of 0.75 and the respective lambda value was 0.001374391, which was found in *Figure 2.2* (see appendix). We derived this from the left most vertical dotted line as that is the value that had the lowest cross validation accuracy (as opposed to the lambda value one standard error from the lambda value that produced the minimum cross validation error), since we are prioritizing model accuracy over interpretability. In *Figure 2.3* (see appendix) we can see the final elastic net model. Below is the mathematical formation of the model's penalty term, which is the only difference between elastic net and multiple linear regression.

$$Penalty\,Term = \lambda\left((1-\alpha)\sum_{j=1}^{p}\beta_j^2 + \alpha\sum_{j=1}^{p}\left|\beta_j\right|\right)$$

α and λ are hyperparameters we set to determine how much of the model is based off the two parent models (Lasso and Ridge) and how strong the overall penalization term is respectively. In combination, these parameters determine how strictly the model is regularized in so that it can generalize to unseen data. Additionally, the first term is the Ridge Regression penalty, and the second term is the Lasso Regression penalty.

```
Call:
glm(formula = Alzheimer.s.Diagnosis ~ Age +
Genetic.Risk.Factor..APOE.ε4.allele. +
   Family.History.of.Alzheimer.s +
Country.Russia.India.Brazil.Mexico.SouthAfrica +
   Urban.vs.Rural.Living + Income.Level.Medium, family = binomial("probit"),
   data = data.train.bin)

Coefficients:
                                          Estimate Std. Error  z value
(Intercept)                             -4.1958252  0.0386153 -108.657
Age                                      0.0491162  0.0004895  100.342
Genetic.Risk.Factor..APOE.ε4.allele.Yes  0.7675531  0.0148757   51.598
Family.History.of.Alzheimer.sYes         0.4984689  0.0128564   38.772
Country.Russia.India.Brazil.Mexico.SouthAfrica  0.3535270  0.0135051   26.177
Urban.vs.Rural.LivingUrban              -0.0199766  0.0117880   -1.695
Income.Level.Medium                      0.0188980  0.0125232    1.509
                                        Pr(>|z|)
(Intercept)                              <2e-16 ***
Age                                      <2e-16 ***
Genetic.Risk.Factor..APOE.ε4.allele.Yes  <2e-16 ***
Family.History.of.Alzheimer.sYes         <2e-16 ***
Country.Russia.India.Brazil.Mexico.SouthAfrica   <2e-16 ***
Urban.vs.Rural.LivingUrban                0.0901 .
Income.Level.Medium                       0.1313
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 75557  on 55712  degrees of freedom
Residual deviance: 60348  on 55706  degrees of freedom
AIC: 60362

Number of Fisher Scoring iterations: 4
```

Figure 2.1 - Summary of probit model

E. Random Forrest

Moving on to more recent decision tree-based models, we decided to fit a Random Forest model. With this uncorrelated tree-based method we decided to run five-fold cross validation with the number of trees set to 100. Initially the model was considerably overfit with a training accuracy of 91.20% and a testing accuracy of 70.02%, so we implemented some regularization parameters to fix this issue:

- Mtry: Determines how many variables are selected as candidates for each potential split, of which we tested 0-10 as the potential values.
- Number of Trees: Determines the total number of trees in the model, of which we tested using 20, 100, 250, and 500 as potential values.
- Max Tree Depth: Determines how many layers of splits are in each tree or the "depth" of the model, of which we tested using 3, 5, 7, 10, 20 as potential values.

Once we tested the wide range of regularization parameters, we used the Gini Impurity of each node as the importance/target metric as it calculates how likely a randomly chosen element from a node is incorrectly labelled. This metric is also widely used in the field for evaluating random forest models and thus is a great metric for us to use in our evaluation process. Our final model we set Mtry to 8, the number of trees to 500, and the max tree depth to 7 to obtain a training and testing accuracy of 73.08% and 72.26% respectively. These accuracies showcase how the model is of good quality and isn't overfit as both accuracies are within 0.85% of each other. Additional model details of the model's performance can be seen in *Table 2.9* (see appendix). You can also find the feature importance levels in *Figure 2.8* (see appendix), but since we are focusing on model performance as opposed to interpretability, we will not cover those findings here.

F. Bagging Model

While we have an uncorrelated tree-based model, we also hoped to implement a correlated tree-based model by using Bagging (Bootstrap Aggregating). Here we ran fivefold cross validation a total of three times with each model to ensure the model fitting process is robust. The initial model was overfit to the training set, with a training and testing accuracy of 92.01% and 71.09% respectively. To prevent the model from overfitting we implemented the following regularization parameters:

- Cost Complexity: A parameter that punishes the model if it becomes too complex, effectively pruning th tree. We used values 0.001. 0.01, 0.1, and 0.25 in the fitting process. The final model used a Cost Complexity of 0.01.
- Minimum Split: The minimum number of observations in a node required to conduct a split. We used values 10, 20, 50.
- Max Tree Depth: Determines the maximum number of layers/sequential splits allowed in a given tree. We used values 3, 5, 7, 10, 20.

Minimum Split of 20, and Max Tree Depth of 5 to obtain a training testing accuracy of 72.79% and 72.39% respectively, demonstrating the model isn't overfit and performs well. Further details of the feature importance and model performance can be found in *Figure 3.0 & Table 3.1* respectively.

```
## Confusion Matrix and Statistics
 ##
##          Reference
 ## Prediction   X0   X1
 ##         X0 7862 2097
 ##         X1 3030 5581
 ##
##               Accuracy : 0.7239
##                 95% CI : (0.7174, 0.7303)
 ##    No Information Rate : 0.5865
##    P-Value [Acc > NIR] : < 2.2e-16
##
##                  Kappa : 0.4408
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
 ##            Sensitivity : 0.7269
##            Specificity : 0.7218
##         Pos Pred Value : 0.6481
##         Neg Pred Value : 0.7894
##             Prevalence : 0.4135
##         Detection Rate : 0.3005
##   Detection Prevalence : 0.4637
##      Balanced Accuracy : 0.7243
##
##       'Positive' Class : X1
##
```

Table 3.1 - Confusion matrix for bagging model test set

VII. Model Evaluation

The confusion matrices for the reduced probit model are given in *Figures 2.4 & 2.5* (see appendix) for the training and testing sets respectively. Here we can see that the accuracy of the model is around 71.39% with the training data set and 71.06% with the testing data set. The proximity of these two values with each other is a good sign that the model is not overfitting the training data set and can predict unseen dataset with similar accuracy. We also have the respective confusion matrices for the elastic net regression model on the training and testing sets in *Figures 2.6 & 2.7* (see appendix), highlighting respective accuracies of 71.62% and 71.25%. This remarkably similar model performance on the training

and testing set shows that the elastic net model is not overfit and should be able to predict the class of unseen data samples with similar accuracy. Additionally, we fit two tree-based models, random forest and bagging, with a training accuracy 73.08% and testing accuracy of 72.26%, as well as a Bagging model with a training accuracy of 72.79% and testing accuracy of 72.39%. Both models are not overfit and perform well as mentioned in previous sections, with additional insights available into the confusion matrices in *Tables 2.8 & 2.9* (see appendix).

VIII. Preliminary Insights

From the six models we have built and performed feature selection on, it seems that most of the lifestyle predictors drop away and the scientific/medical predictors such as history of Alzheimer's, age, and the APOE e4 allele survive higher penalties. It makes logical sense that the medical risk factors would be able to better predict the diagnosis of Alzheimer's.

IX. Future Studies and Model Maintenance

Maintaining the model is important, so we have devised a plan on how to accomplish this:

1. First, we will continually look for more data that can be included in this dataset to expand the applicability of our findings. For example, as data from more countries become available, we will be sure to include these data in our evaluations.
2. Second, we will be proactively testing and measuring the data with more testing techniques, especially as more data are discovered.
3. Finally, we will be performing new modeling techniques as they become available.

We will be through in discovering where and how we can collect new samples in the future. This may include going to hospitals and obtaining the personal information of patients while removing all personally identifiable information. We can also collect more granular data, for example collecting if the patient has any other mental disorders/illnesses.

X. Conclusion

We have fit six different models of varying accuracies but the most optimal versions of each have a testing accuracy between 70-73%, with the most accurate model being the regularized bagging model with a testing accuracy of 72.39% which is great performance for predicting whether a patient has Alzheimer's based on these few predictors. Though our primary goal was accuracy, we can still learn from the Bagging model that the most important predictors are Age, presence of the APOE e4 allele, family history of Alzheimer's, and country in that respective order as seen in *Figure 3.0*.

We hope that this model will be used in a variety of applications. For example, an online quiz hosted by a healthcare provider or insurance company could use the model to recommend whether a patient should seek further medical testing. Additionally, the model can simulate the age at which someone is predicted to develop Alzheimer's by gradually increasing the input age while holding all other factors constant, offering both patients and providers a statistical estimate of risk onset. This could prove useful for designing personalized intervention timelines and motivating individuals to adopt healthier habits, especially if modifiable factors such as diet, sleep, and exercise influence their predicted trajectory.

However, it should be noted that this model cannot and should not be used interchangeably with a medical diagnosis. False positives and false negatives are certainly still possible in when using this model, so it should be addressed as such. Additionally, while we exercised caution when deciding which features to include in analysis so as to not violate any ethical concerns, this same caution must be demonstrated as the model is deployed into the real world. Concerns like data privacy and equity across different demographic groups must be considered when, for example, the model may be used to control an online quiz. Thus, if used with ethical caution, our final model can be used to further public health and prevent Alzheimer's disease.

References

[1] Panday, A. (2024, August 30). Alzheimer's Prediction dataset (global). Kaggle. https://www.kaggle.com/datasets/ankushpanday1/alzheimers-prediction-dataset-global/data J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

Section 0 - Introduction to the Data

```
 [1] "Country"
 [2] "Age"
 [3] "Gender"
 [4] "Education.Level"
 [5] "BMI"
 [6] "Physical.Activity.Level"
 [7] "Smoking.Status"
 [8] "Alcohol.Consumption"
 [9] "Diabetes"
[10] "Hypertension"
[11] "Cholesterol.Level"
[12] "Family.History.of.Alzheimer.s"
[13] "Cognitive.Test.Score"
[14] "Depression.Level"
[15] "Sleep.Quality"
[16] "Dietary.Habits"
[17] "Air.Pollution.Exposure"
[18] "Employment.Status"
[19] "Marital.Status"
[20] "Genetic.Risk.Factor..APOE.ε4.allele."
[21] "Social.Engagement.Level"
[22] "Income.Level"
[23] "Stress.Levels"
[24] "Urban.vs.Rural.Living"
[25] "Alzheimer.s.Diagnosis"
```

Figure 0.1 - List of all variables in the dataset.

Section 1 - Exploratory Data Analysis

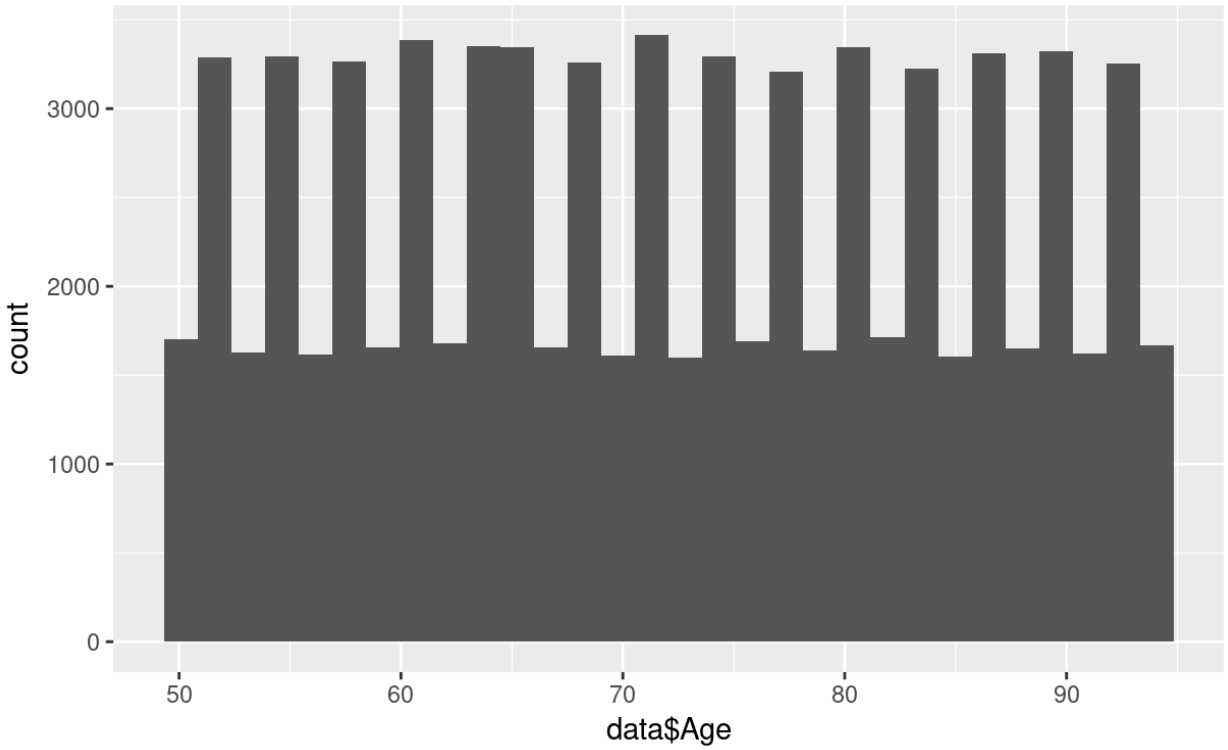

Figure 1.1 The bar chart shows Age plotted against its frequency. The distribution is not normally distributed.

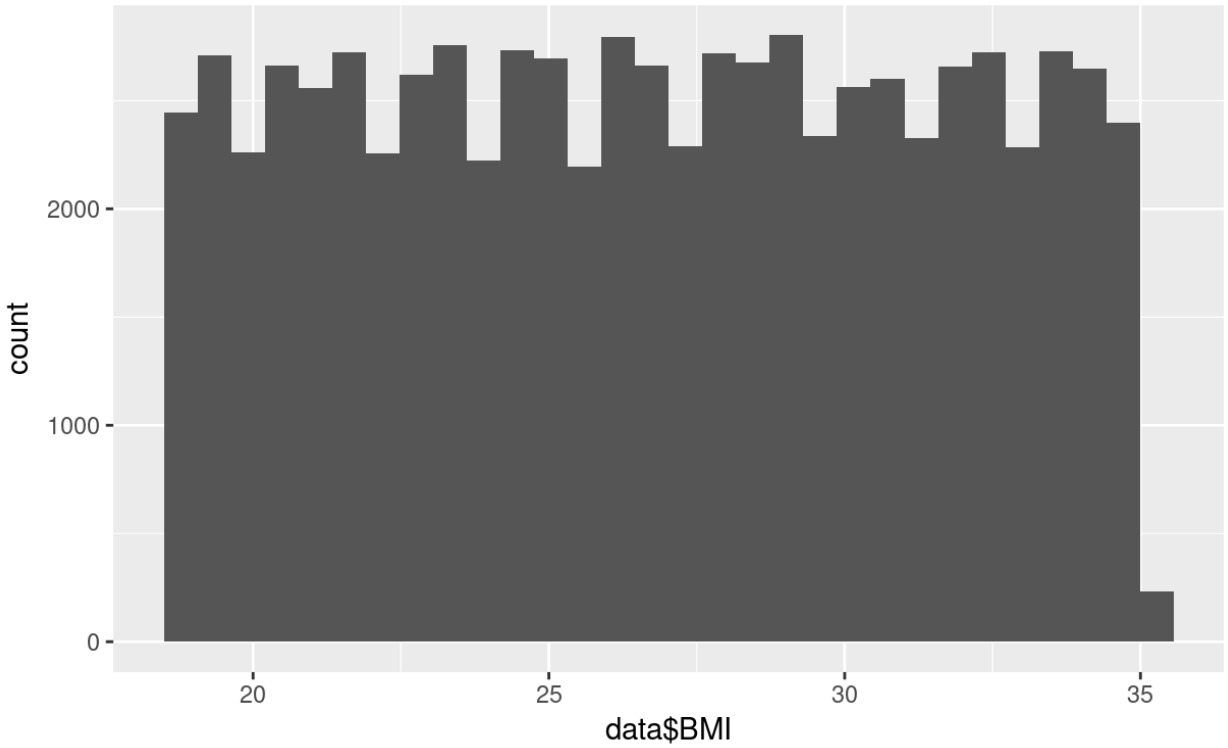

Figure 1.2 - The bar chart shows DMI plotted against its frequency. The distribution is not normally distributed.

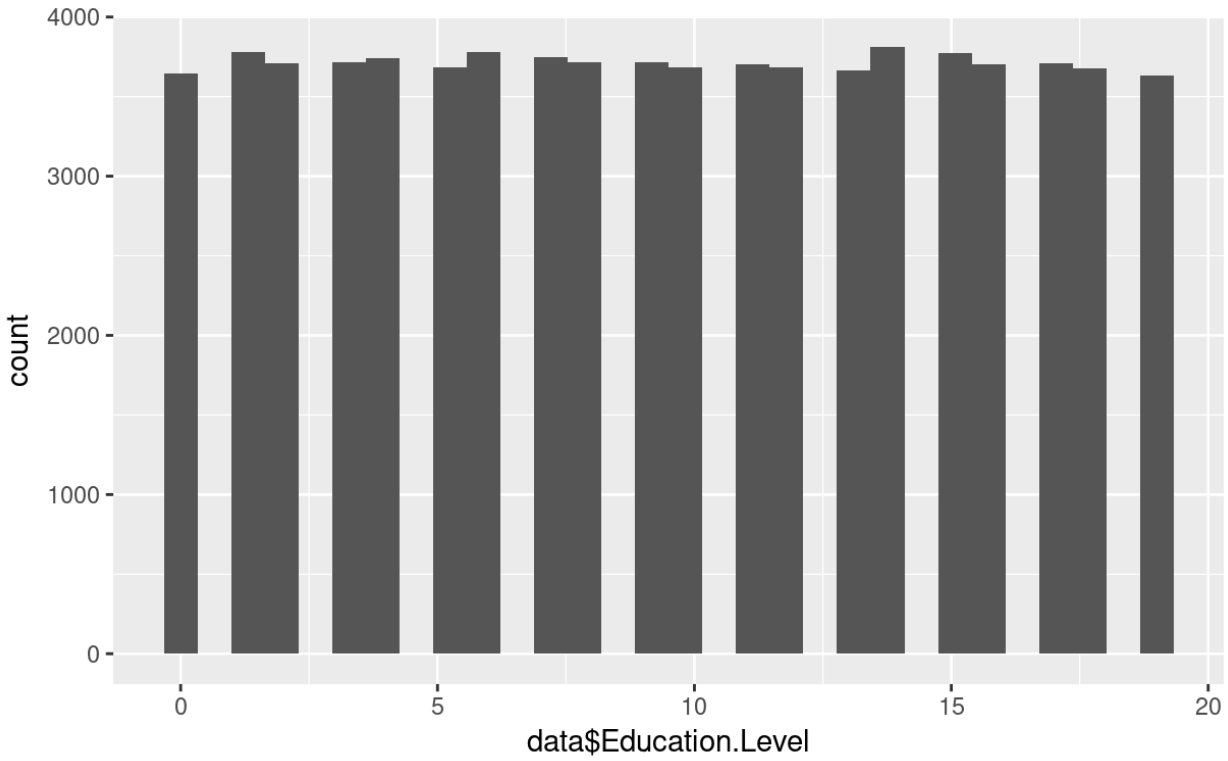

Figure 1.3 - The bar chart shows Education Level plotted against its frequency. The distribution is not normally distributed.

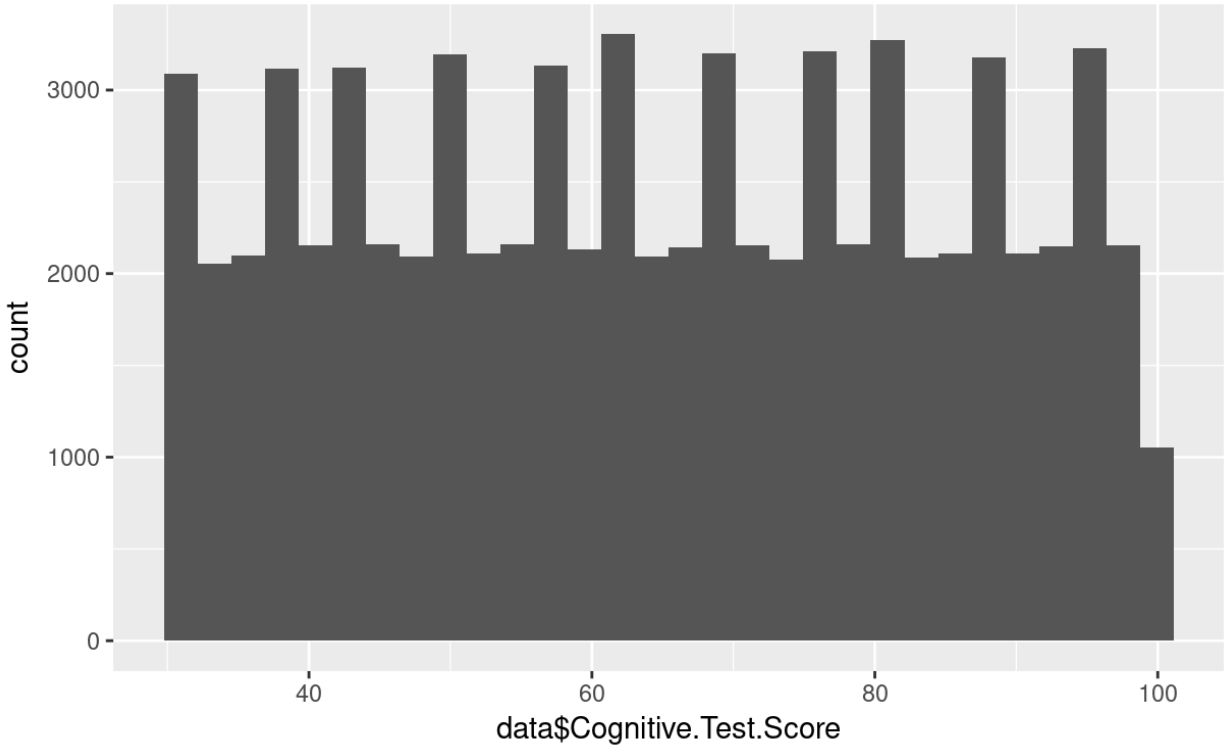

Figure 1.4 - The graph shows Cognitive Test Score plotted against its frequency. The distribution is not normally distributed.

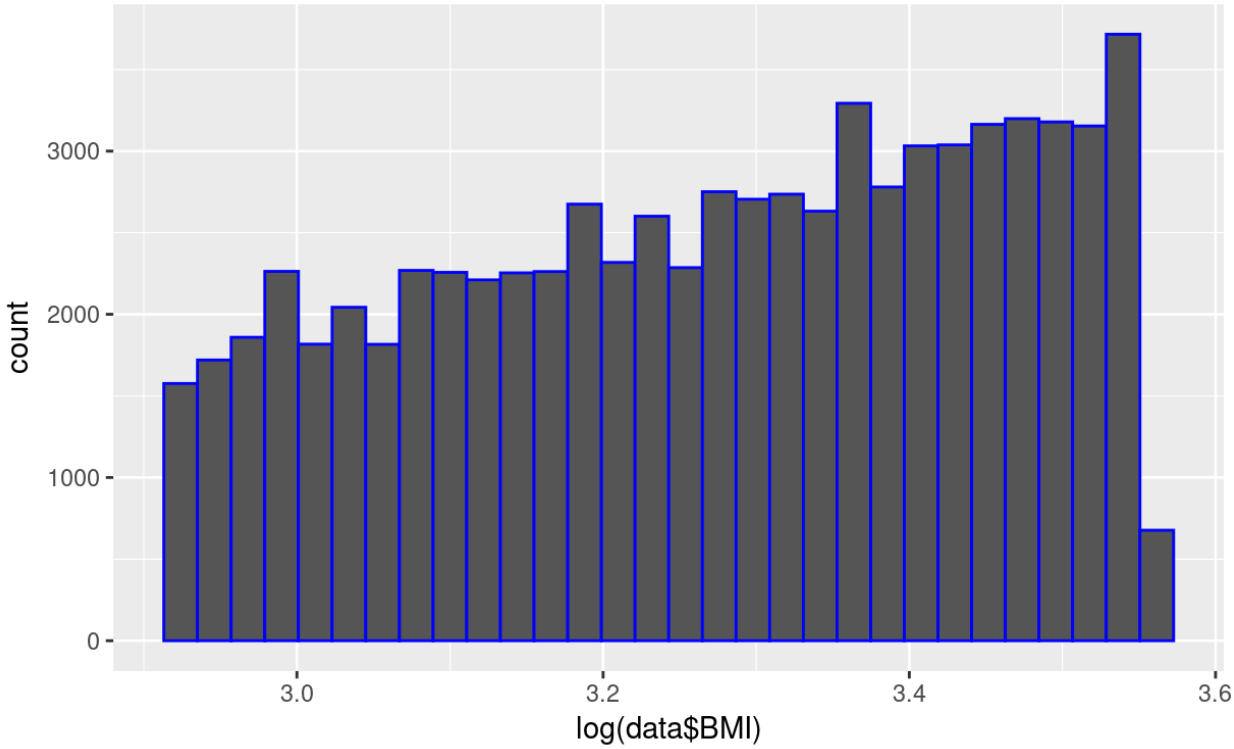

Figure 1.5 - The bar chart shows BMI plotted against its frequency with the log transformation applied. The distribution is not normally distributed.

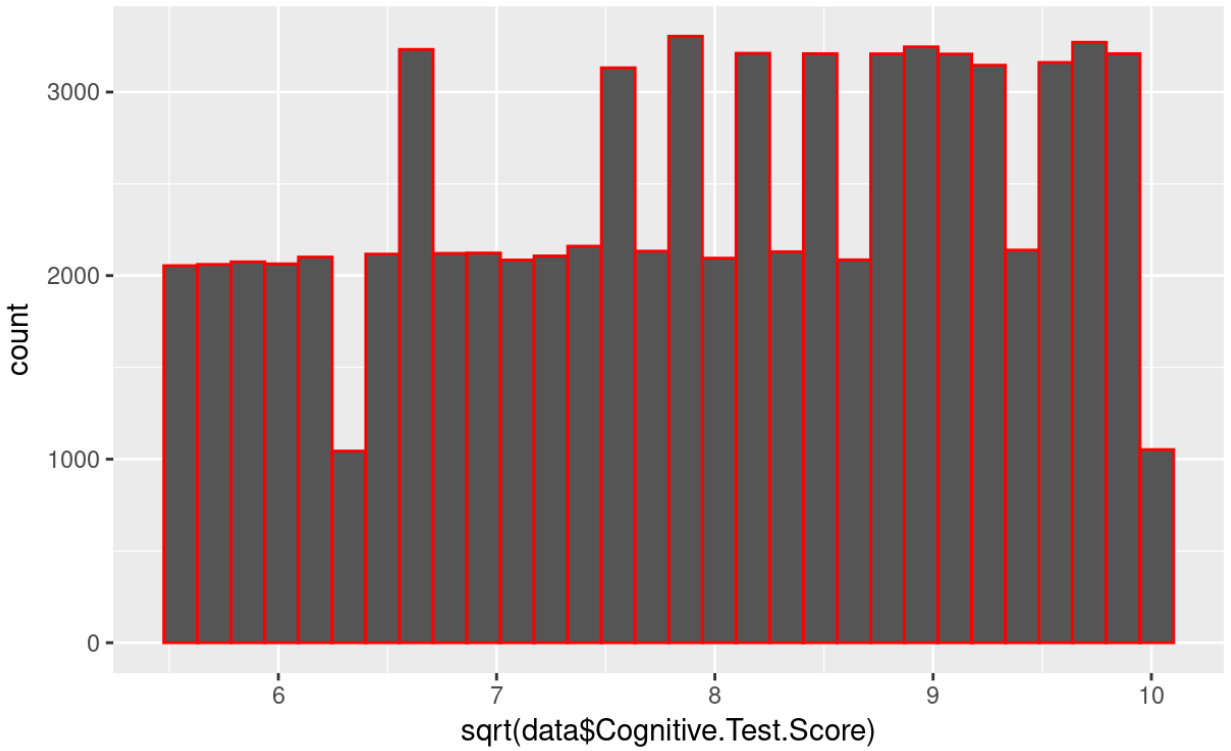

Figure 1.6 - The bar chart shows Cognitive Test Score plotted against its frequency with the square-root transformation applied. The distribution is not normally distributed.

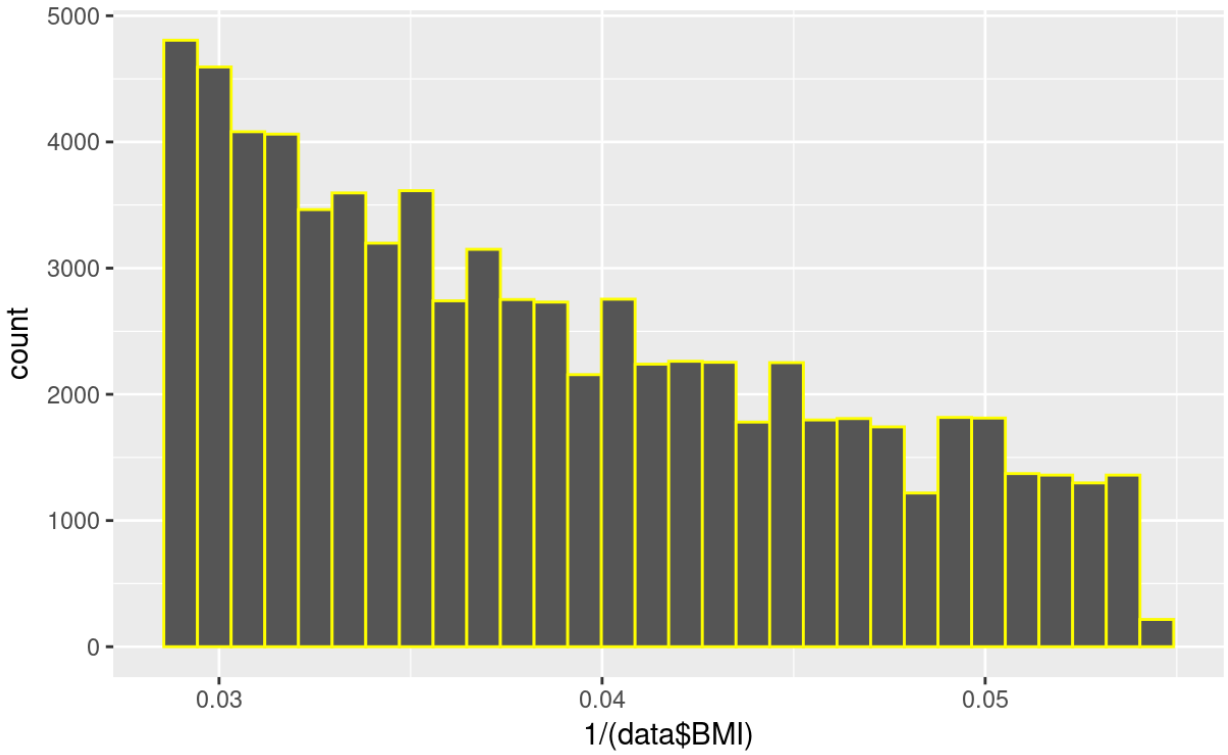

Figure 1.7 - The bar chart shows BMI plotted against its frequency with the inverse transformation applied. The distribution is not normally distributed.

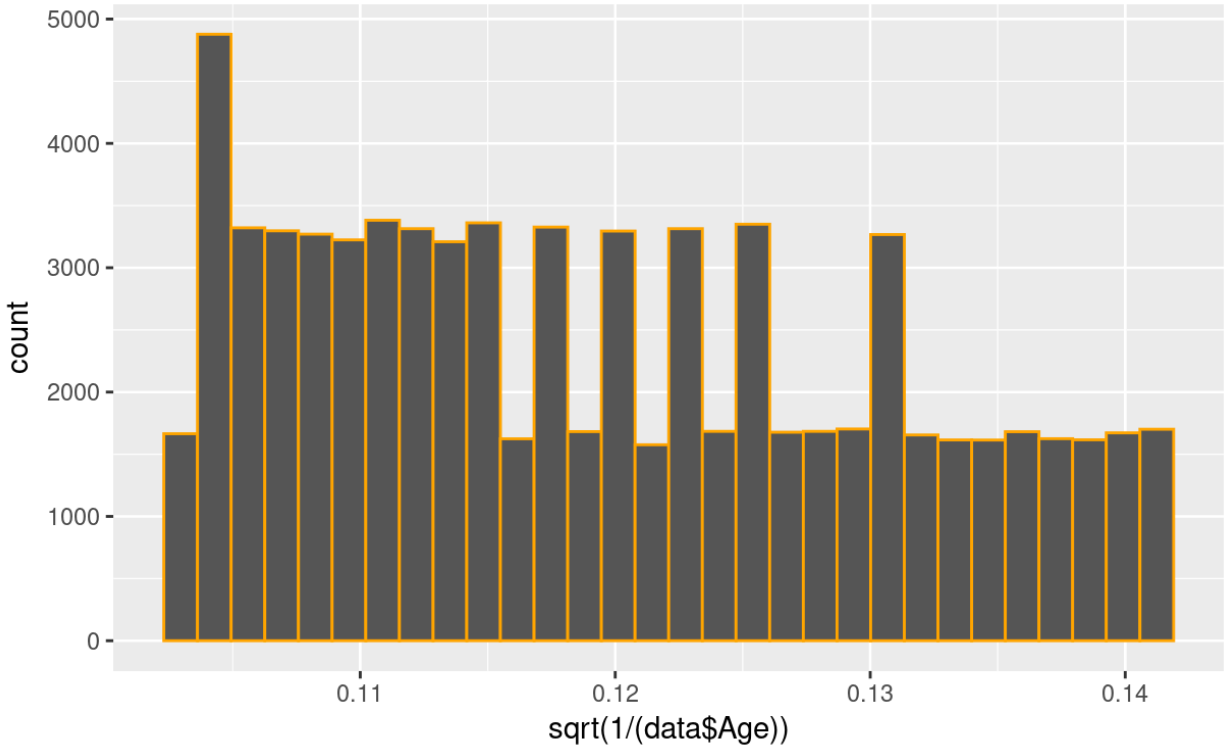

Figure 1.8 - The bar chart shows Age plotted against its frequency with the square-root of the inverse transformation applied. The distribution is not normally distributed.

Figure 1.9 - The bar chart shows Cognitive Test Score plotted against its frequency with the log of the inverse transformation applied. The distribution is not normally distributed.

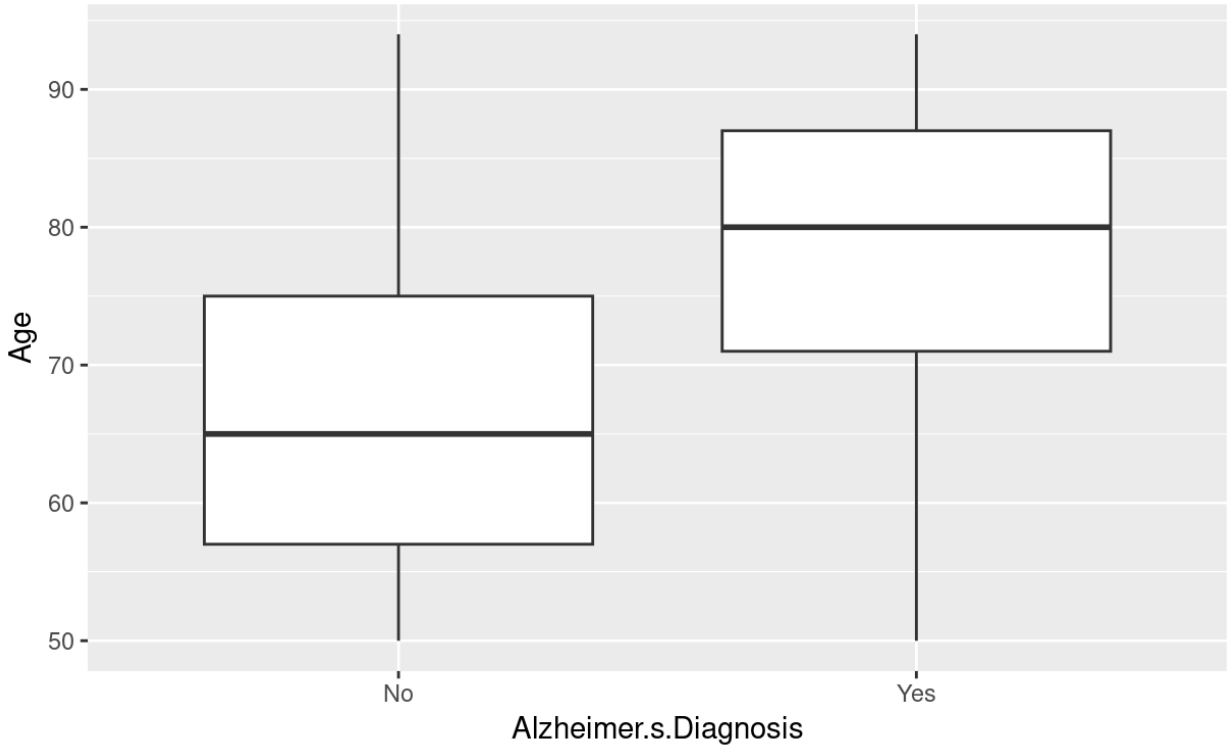

Figure 1.10 - The boxplot shows Age separated by Alzheimer's Diagnosis. The means between the two categories of diagnoses are significantly different. Therefore, Age is a good predictor of Alzheimer's.

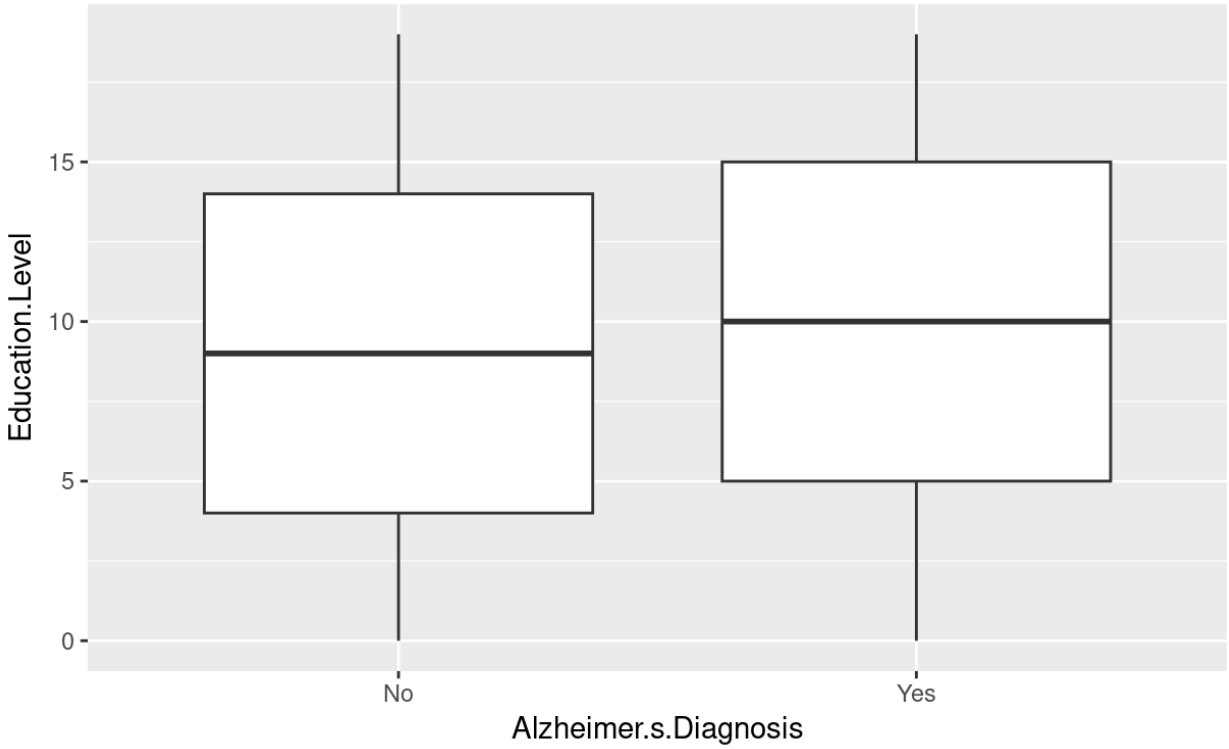

Figure 1.11 - The boxplot shows Education Level separated by Alzheimer's Diagnosis. The means between the two categories of diagnoses are not significantly different. Therefore, Education Level is not a good predictor of Alzheimer's.

```
## # A tibble: 2 × 4
##   Alzheimer.s.Diagnosis  mean median     n
##   <fct>                  <dbl>  <dbl> <int>
## 1 No                      67.4     65 43570
## 2 Yes                     78.5     80 30713
```

Table 1.1 Means of Age grouped by Alzheimer's Diagnosis. The means are significantly different, so Education is a good predictor of Alzheimer's.

```
## # A tibble: 2 × 4
##   Alzheimer.s.Diagnosis  mean median     n
##   <fct>                  <dbl>  <dbl> <int>
## 1 No                      9.47      9 43570
## 2 Yes                     9.51     10 30713
```

Table 1.2 Means of Education grouped by Alzheimer's Diagnosis. The means are not significantly different, so Education is not a good predictor of Alzheimer's.

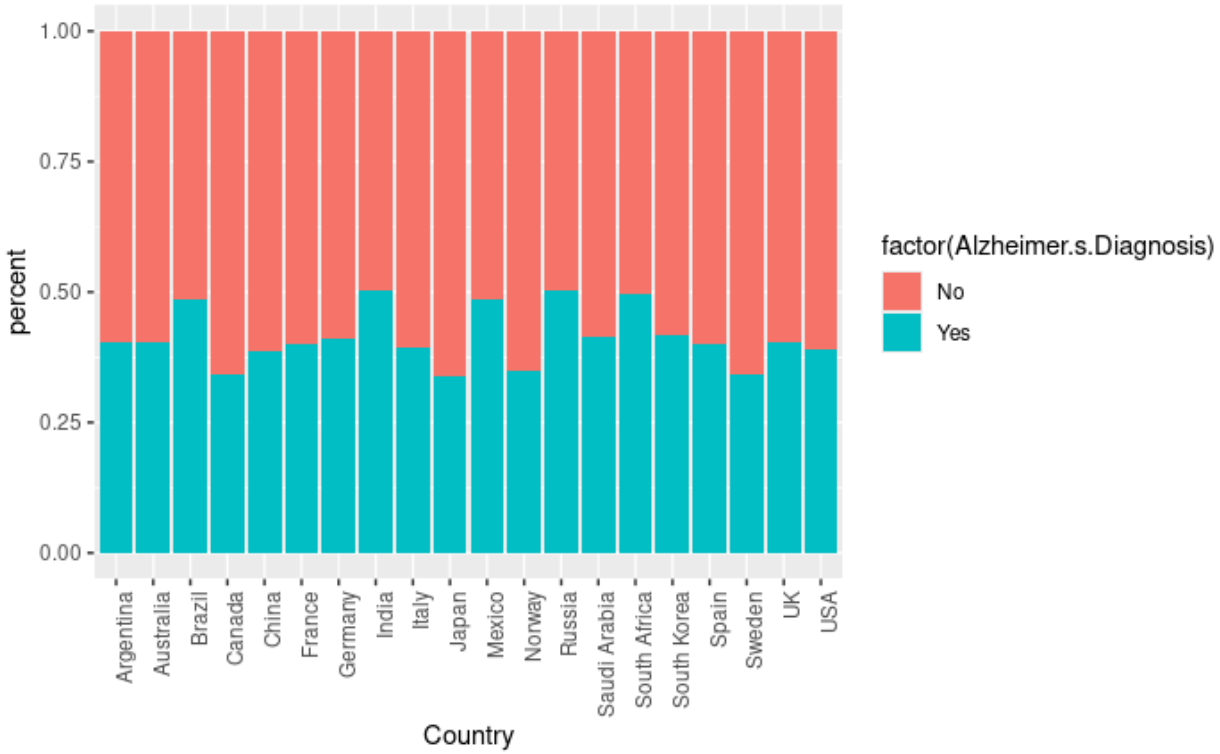

Figure 1.12.1 - The bar chart shows the proportion of people with Alzheimer's by country. The proportions are significantly different from each other.

```
## # A tibble: 20 × 3
##         Country              mean     n
##         <fct>     <dbl> <int>
##  1 Argentina    0.402  3731
##  2 Australia    0.403  3787
##  3 Brazil       0.486  3839
##  4 Canada       0.341  3711
##  5 China        0.386  3592
##  6 France       0.401  3710
##  7 Germany      0.411  3807
##  8 India        0.503  3741
##  9 Italy        0.393  3724
## 10 Japan        0.339  3751
## 11 Mexico       0.485  3598
## 12 Norway       0.350  3706
## 13 Russia       0.504  3778
## 14 Saudi Arabia 0.414  3662
## 15 South Africa 0.495  3760
## 16 South Korea  0.416  3732
## 17 Spain        0.400  3698
## 18 Sweden       0.342  3689
## 19 UK           0.404  3651
## 20 USA          0.389  3616
```

Table 1.2 - The table shows the means of the Alzheimer's Diagnosis variable for each country. Notice how some are significantly higher than others.

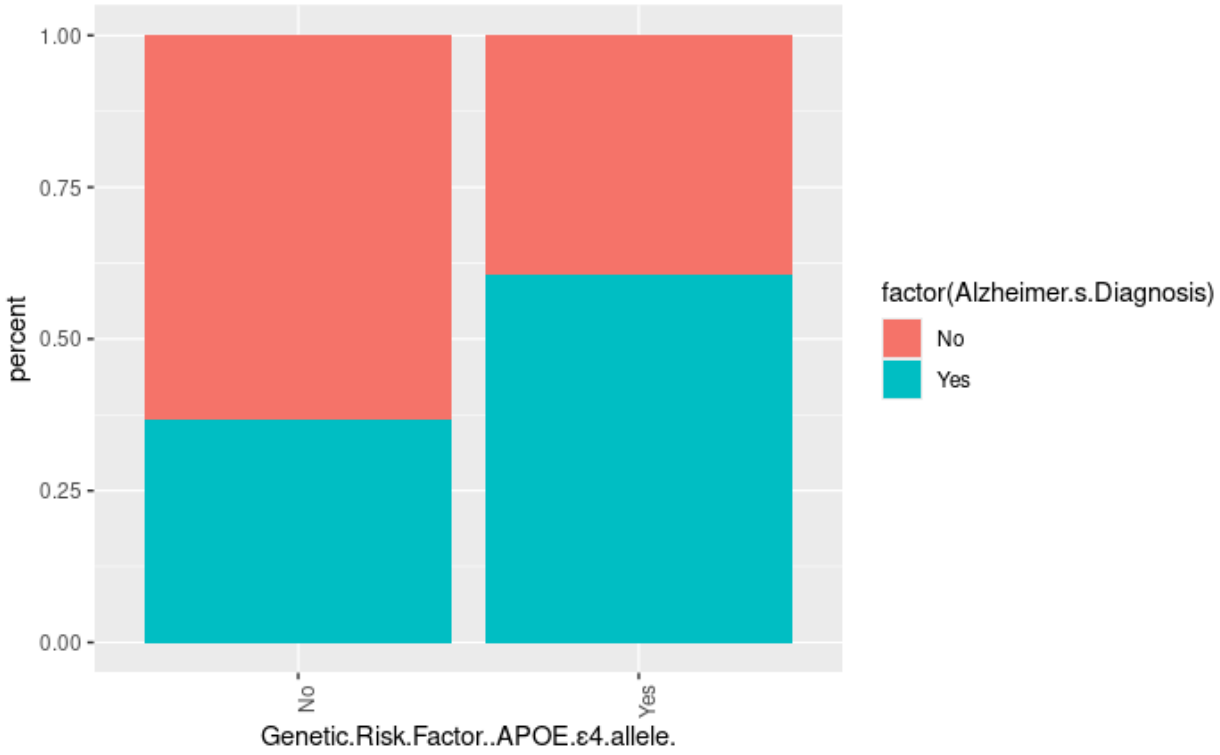

Figure 1.13 - The bar chart shows the proportion of people with Alzheimer's by whether a person has the Genetic Risk Factor. The proportions are significantly different from each other.

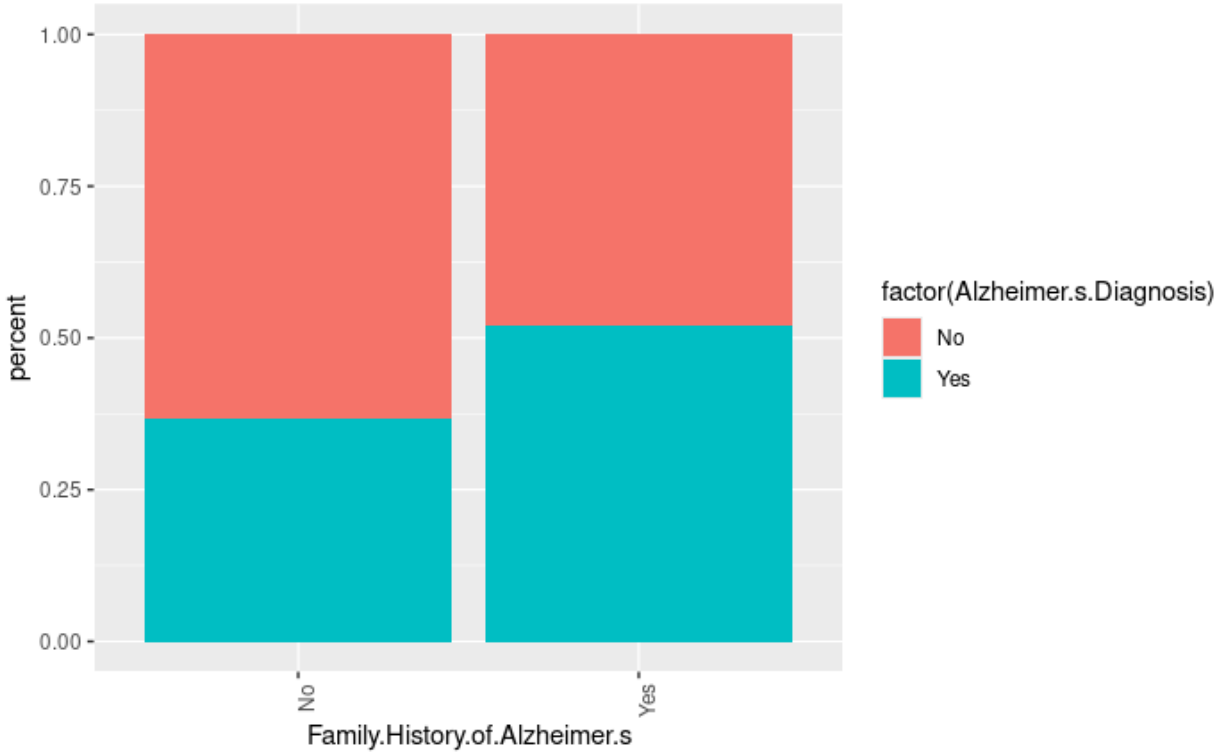

Figure 1.14 - The bar chart shows the proportion of people with Alzheimer's by whether a person has a family history of Alzheimer's. The proportions are significantly different from each other.

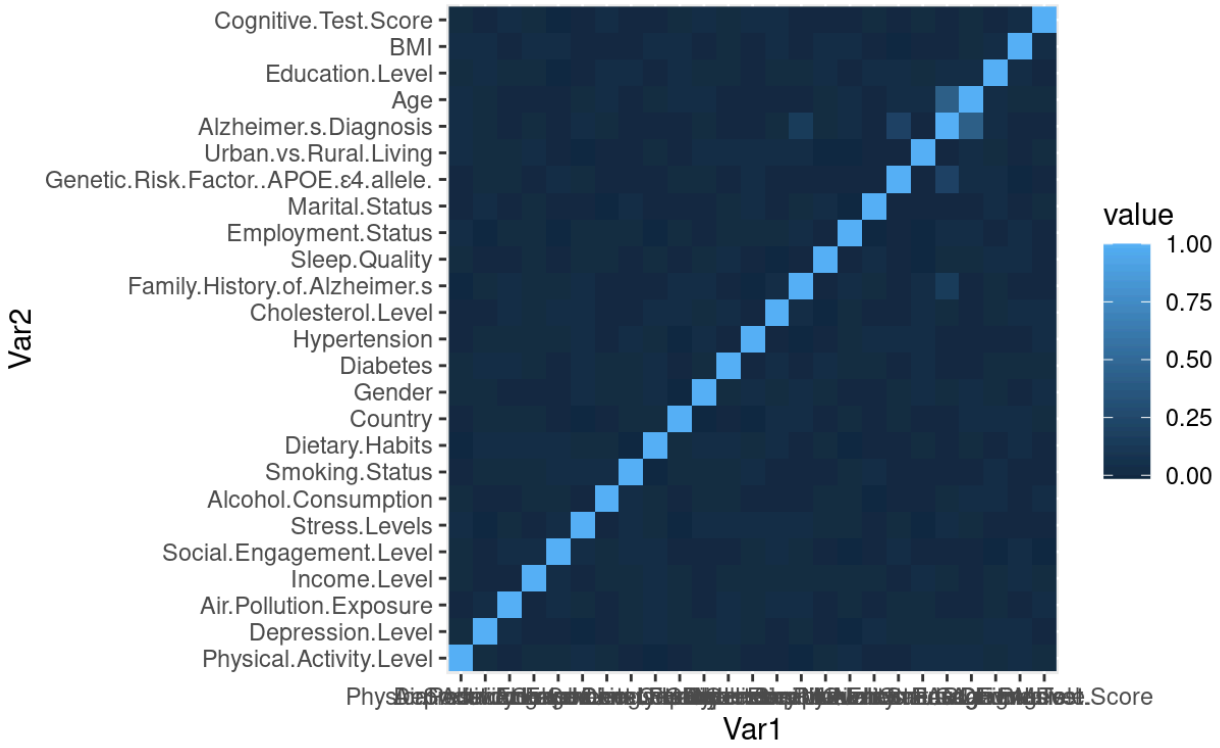

Figure 1.15 - The correlation matrix shows that there is no collinearity as all of the variables that are correlated with the target variable.

```
## # A tibble: 7 × 3
##   Country       mean     n
##   <fct>         <dbl>  <int>
## 1 South America 0.4445178  7570
## 2 Australia     0.4026934  3787
## 3 North America 0.4042105  10925
## 4 Asia          0.4120035  18478
## 5 Europe        0.4009004        25956
## 6 South Africa  0.4952128  3760
```

Table 1.3 - When grouped by continent, the means of each of the countries converge to around 0.4. Thus, the countries should not be grouped by country.

```
## # A tibble: 2 × 3
##   Country                          mean     n
##   <fct>                            <dbl> <int>
## 1 Other                            0.386 55567
## 2 Russia-India-Brazil-Mexico-SouthAfrica 0.495 18716
```

Table 1.4 - When grouping the countries with a mean >0.42 together, the means of each of the country groups are still significantly different. Thus, the countries should be grouped this way.

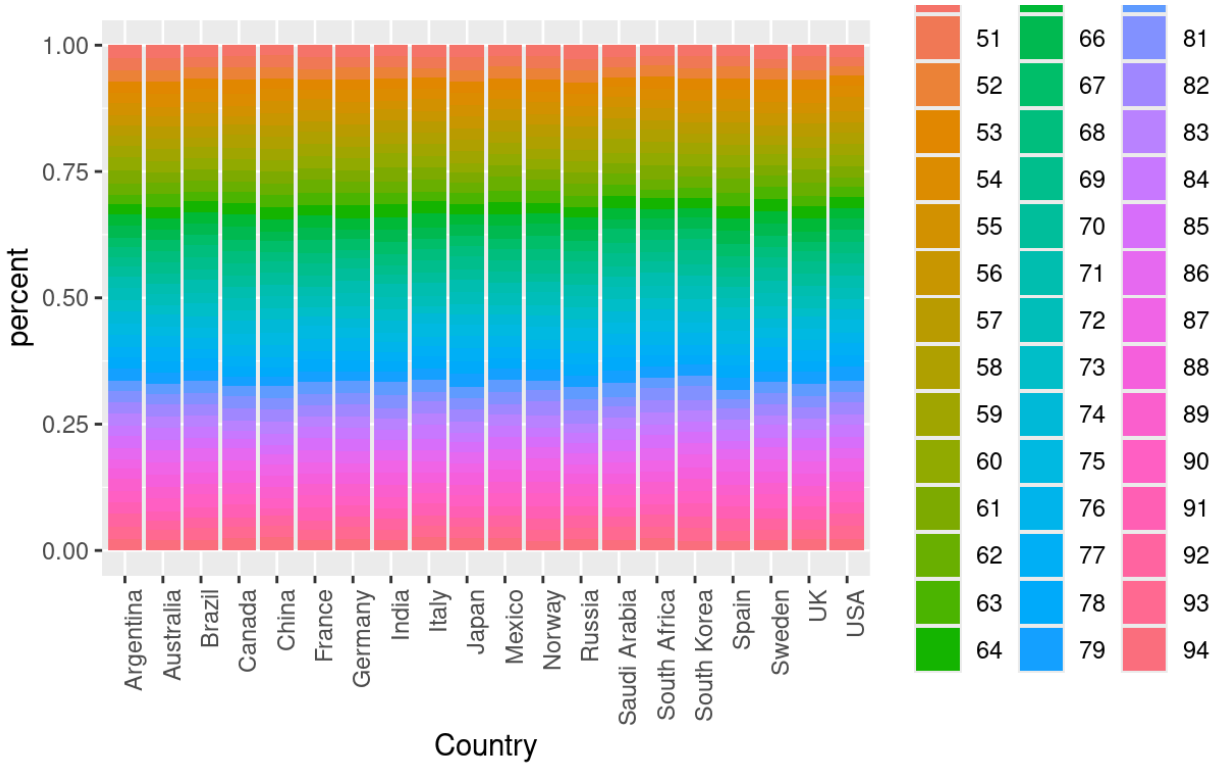

Figure 1.16.1 - Prior to grouping countries with mean >0.42 together, there is some collinearity between country and Age.

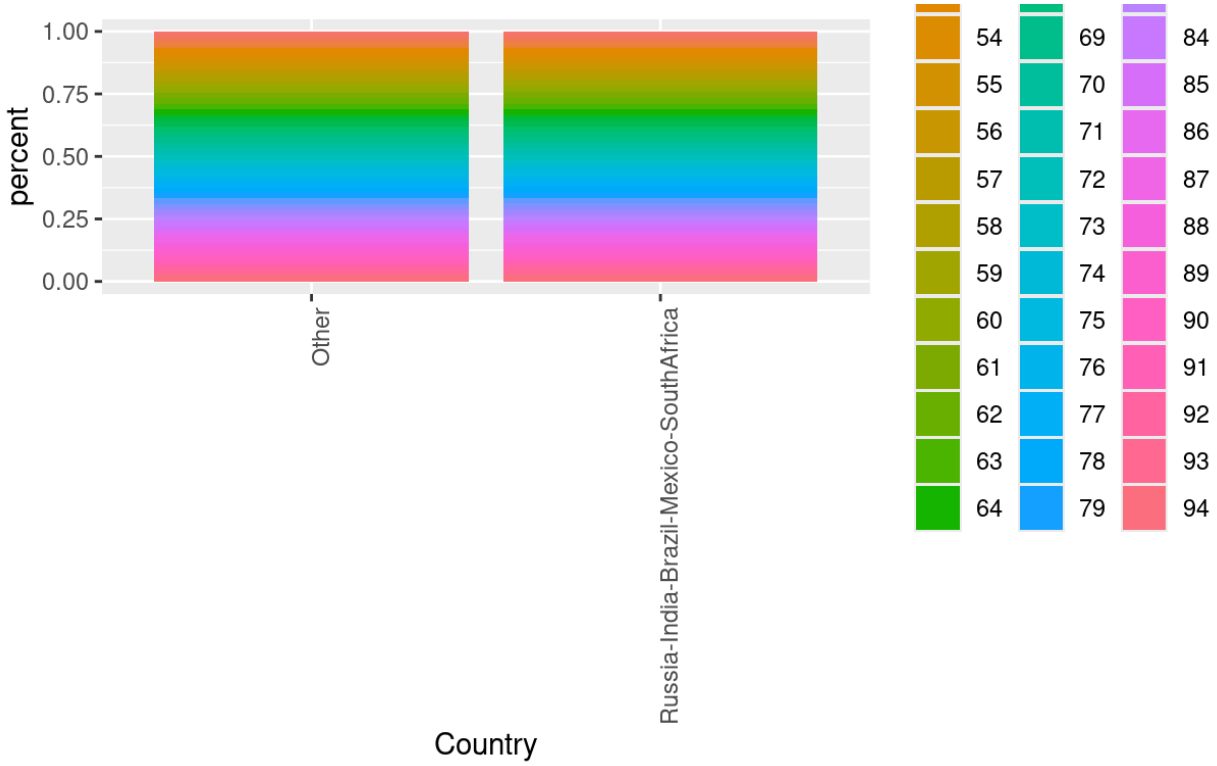

Figure 1.16.2 After grouping countries with mean >0.42 together, there is no collinearity and the dimension has been reduced.

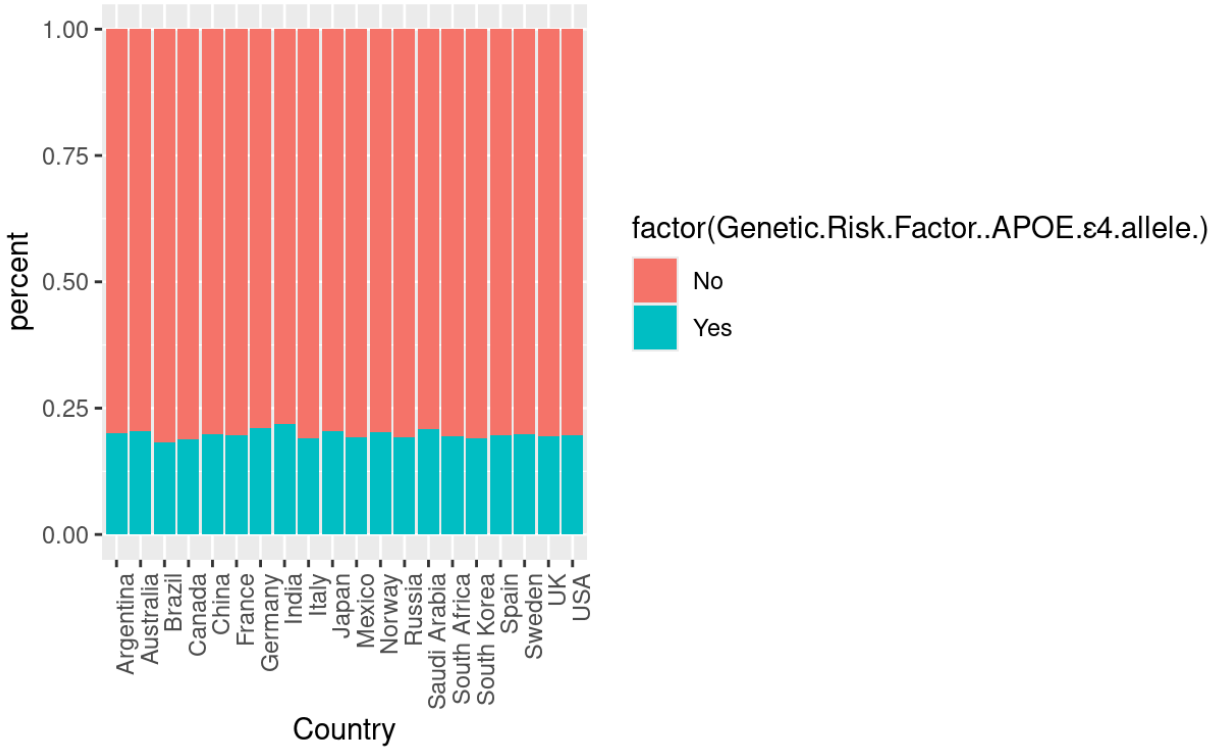

Figure 1.17.1 - Prior to grouping countries with mean >0.42 together, there is some collinearity between country and Genetic Risk Factor.

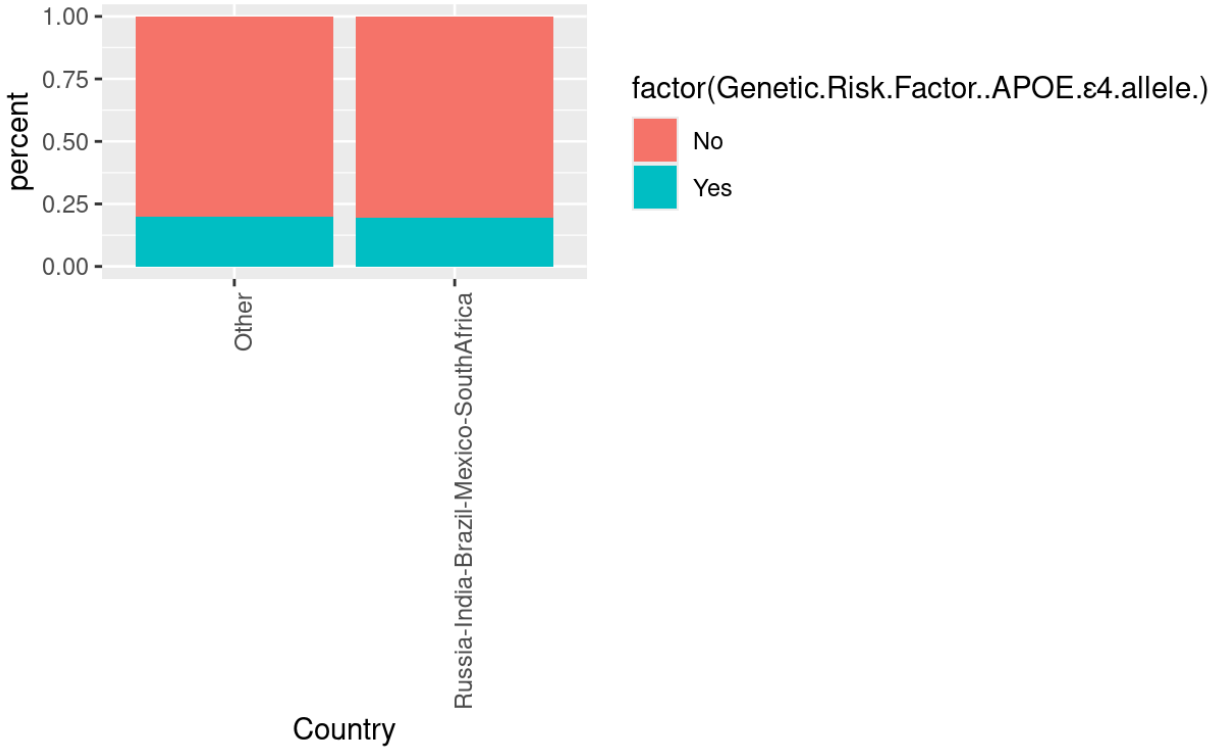

Figure 1.18 - After grouping countries with mean >0.42 together, there is no collinearity and the dimension has been reduced.

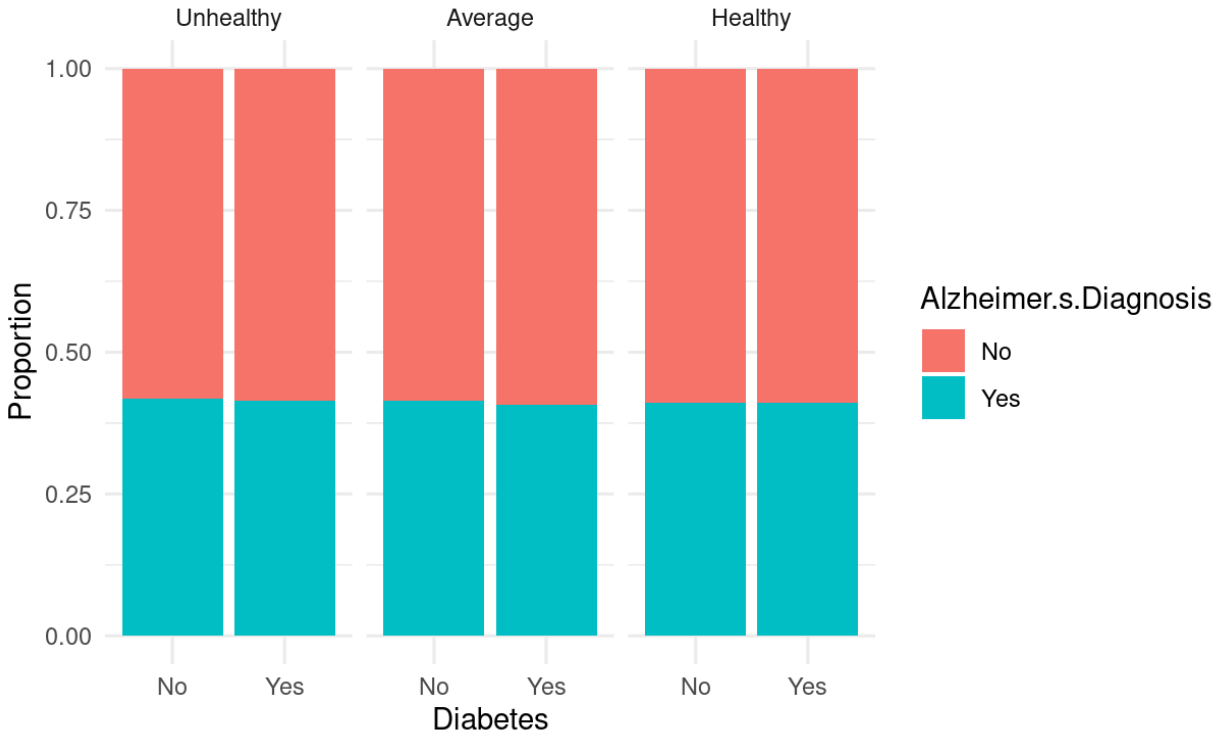

Figure 1.19 - Diet vs Diabetes - To look for interaction between variables, proportion of people with Alzheimer's grouped by Diabetes and faceted by Diet is shown in the bar chart. The means are about equal, so the variables do not interact.

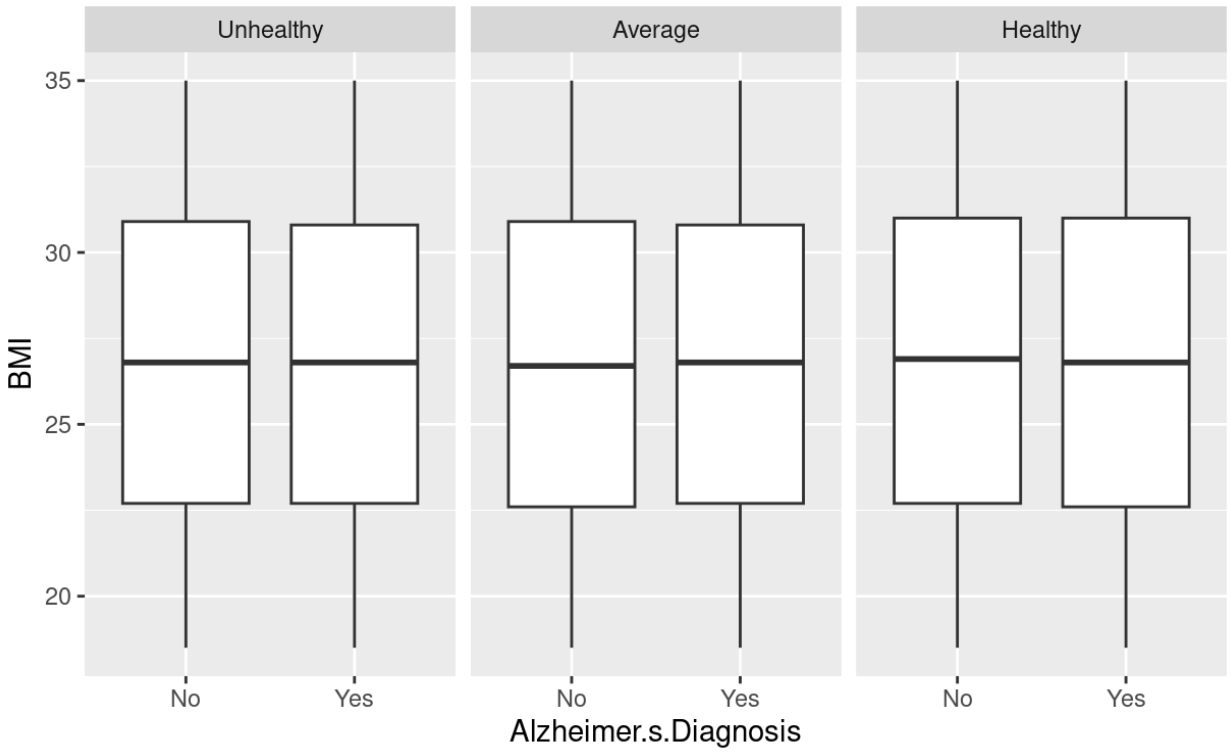

Figure 1.20 - Diet vs BMI - To look for interaction between variables, proportion of people with Alzheimer's grouped by BMI and faceted by Diet is shown in the box-plot. The means are about equal, so the variables do not interact.

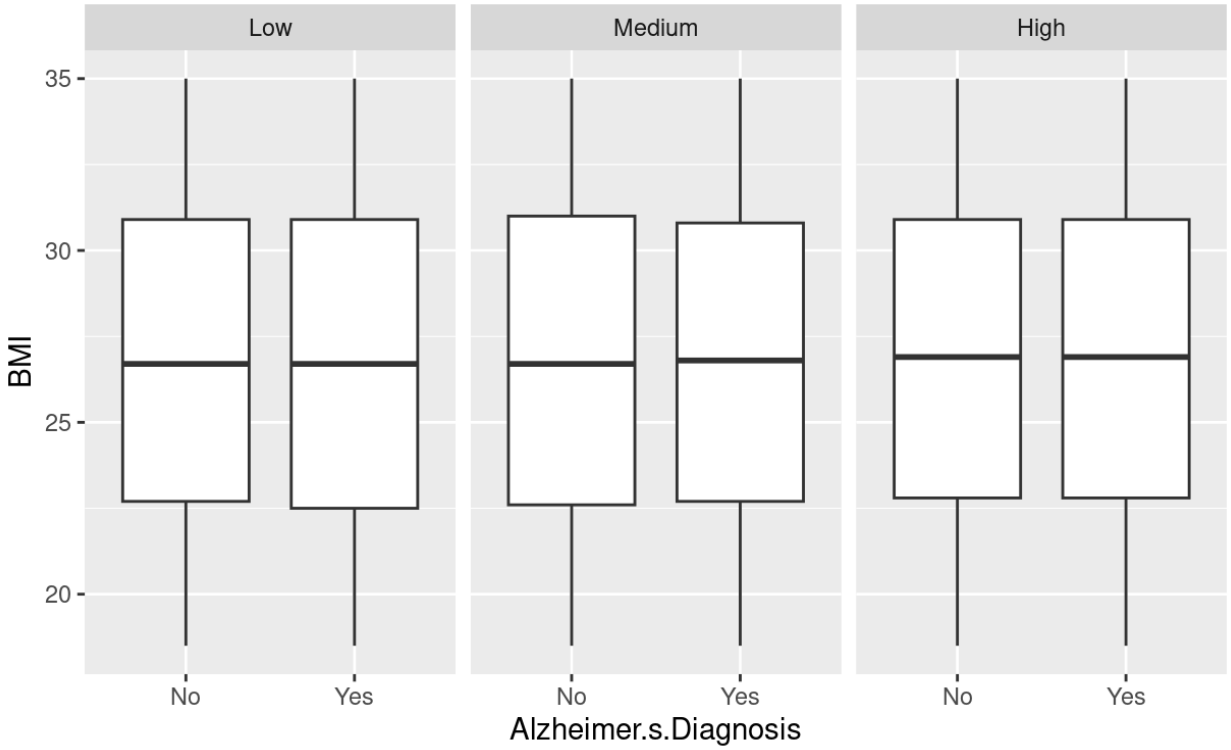

Figure 1.21 - Physical Activity vs BMI- To look for interaction between variables, proportion of people with Alzheimer's grouped by BMI and faceted by Physical Activity Level is shown in the box-plot. The means are about equal, so the variables do not interact.

Section 2 - Model Building Process

```
Call:
glm(formula = Alzheimer.s.Diagnosis ~ Age + Genetic.Risk.Factor..APOE.ε4.allele. +
    Family.History.of.Alzheimer.s + Country.Russia.India.Brazil.Mexico.SouthAfrica +
    Urban.vs.Rural.Living + Income.Level.Medium, family = binomial("probit"),
    data = data.train.bin)

Coefficients:
                                          Estimate Std. Error  z value
(Intercept)                             -4.1958252  0.0386153 -108.657
Age                                      0.0491162  0.0004895  100.342
Genetic.Risk.Factor..APOE.ε4.allele.Yes  0.7675531  0.0148757   51.598
Family.History.of.Alzheimer.sYes         0.4984689  0.0128564   38.772
Country.Russia.India.Brazil.Mexico.SouthAfrica  0.3535270  0.0135051   26.177
Urban.vs.Rural.LivingUrban              -0.0199766  0.0117880   -1.695
Income.Level.Medium                      0.0188980  0.0125232    1.509
                                        Pr(>|z|)
(Intercept)                              <2e-16 ***
Age                                      <2e-16 ***
Genetic.Risk.Factor..APOE.ε4.allele.Yes  <2e-16 ***
Family.History.of.Alzheimer.sYes         <2e-16 ***
Country.Russia.India.Brazil.Mexico.SouthAfrica   <2e-16 ***
Urban.vs.Rural.LivingUrban                0.0901 .
Income.Level.Medium                       0.1313
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 75557  on 55712  degrees of freedom
Residual deviance: 60348  on 55706  degrees of freedom
AIC: 60362

Number of Fisher Scoring iterations: 4
```

Figure 2.1 - Summary of probit model

[1] 60475.75

[1] 68015.67

[1] 60405.24

Figure 2.2 - AIC for the logit base regression model, probit regression model, and logit regression model with an offset respectively.

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction     0     1
##          0 23151  6412
##          1  9527 16623
##
##                Accuracy : 0.7139
##                  95% CI : (0.7101, 0.7177)
##     No Information Rate : 0.5865
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4217
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.7216
##             Specificity : 0.7085
##          Pos Pred Value : 0.6357
##          Neg Pred Value : 0.7831
##              Prevalence : 0.4135
##          Detection Rate : 0.2984
##    Detection Prevalence : 0.4694
##          Balanced Accuracy : 0.7150
##
##          'Positive' Class : 1
##
```

Table 2.1 - Confusion matrix of probit model

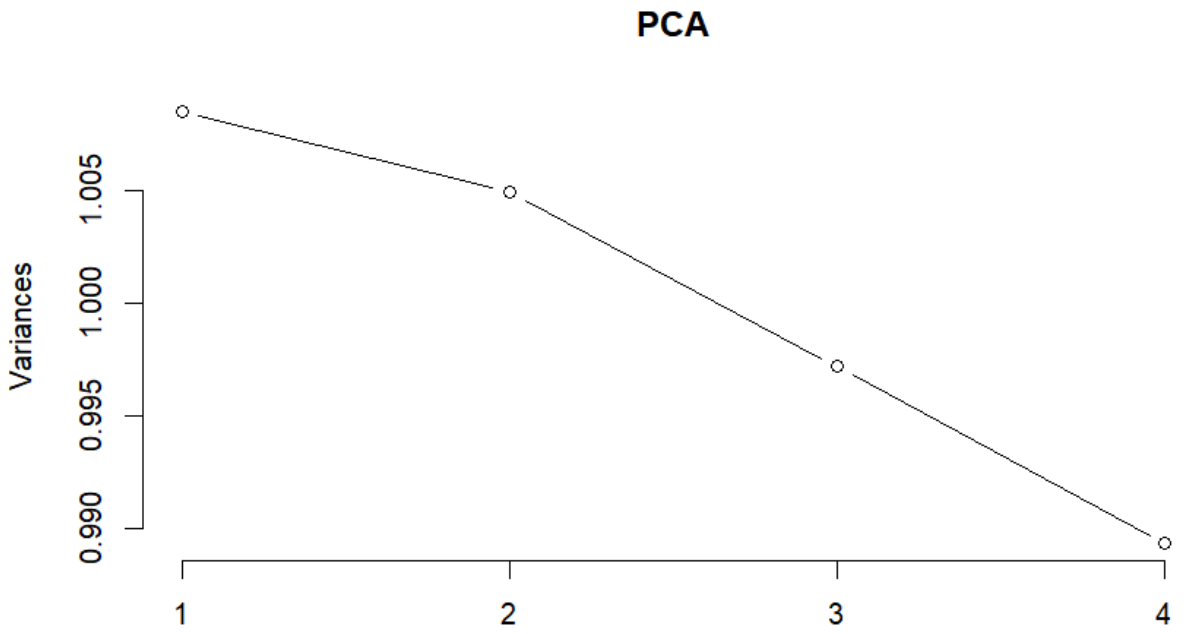

Figure 2.2.1 – Plot of the principal components against their explained variances in the probit model.

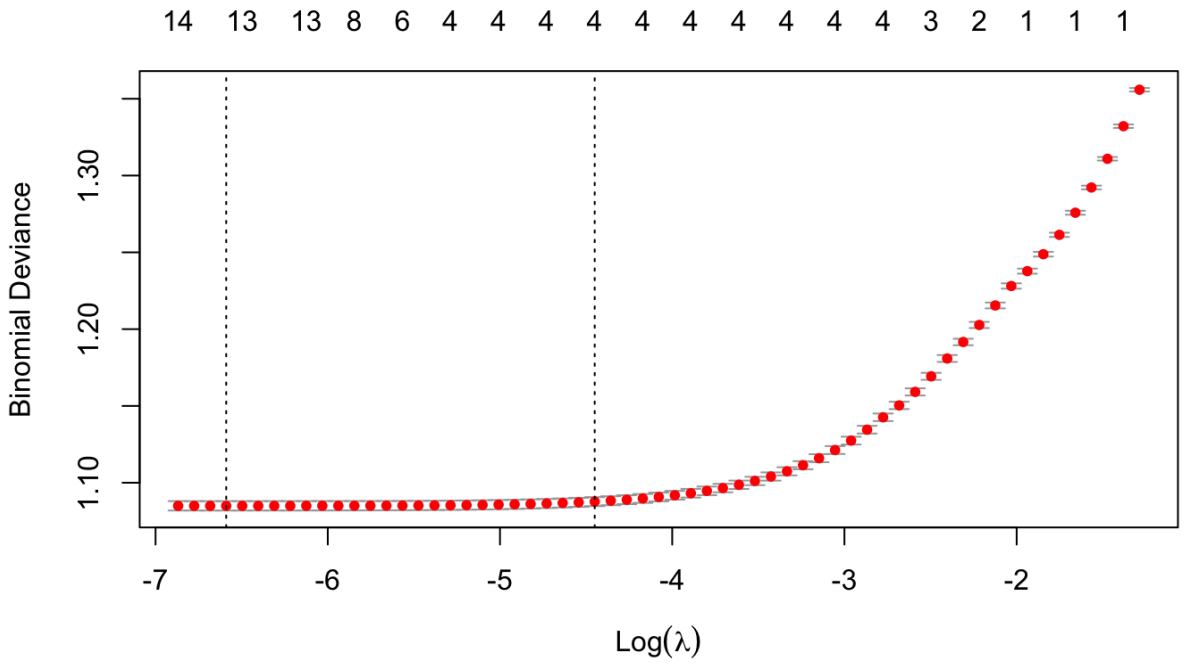

Figure 2.3 Cross Validation plot of Log(lambda) vs Binomial Deviance for alpha = 0.75.

```
##
## Call:  glmnet(x = x_train, y = y_train, family = "binomial", alpha = alpha_val,             lambda = lambda_Search$lambda.min)
##
##   Df  %Dev   Lambda
## 1 13 20.04 0.001374
```

```
## 25 x 1 sparse Matrix of class "dgCMatrix"
##                                        s1
## (Intercept)                    -9.501977265
## Physical.Activity.Level         .
## Depression.Level                .
## Air.Pollution.Exposure         -0.012812816
## Income.Level                    .
## Social.Engagement.Level        -0.004700018
## Stress.Levels                  0.005521337
## Alcohol.Consumption             .
## Smoking.Status                  .
## Dietary.Habits                 -0.008656046
## Country                        0.562905276
## Gender                          .
## Diabetes                        .
## Hypertension                   -0.010629246
## Cholesterol.Level              0.016506077
## Family.History.of.Alzheimer.s              0.805721834
## Sleep.Quality                  0.005820998
## Employment.Status               0.005830286
## Marital.Status                  .
## Genetic.Risk.Factor..APOE.ε4.allele.  1.253380134
## Urban.vs.Rural.Living          -0.021920136
## Age                      0.081058367
## Education.Level                 .
## BMI                             .
## Cognitive.Test.Score            .
```

Table 2.3 - Final Elastic Net Model

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction     0         1
##          0 23151  6412
##          1  9527 16623
##
##                Accuracy : 0.7139
##                  95% CI : (0.7101, 0.7177)
##          No Information Rate : 0.5865
##          P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4217
```

```
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.7216
##          Specificity : 0.7085
##          Pos Pred Value : 0.6357
##          Neg Pred Value : 0.7831
##          Prevalence : 0.4135
##          Detection Rate : 0.2984
##          Detection Prevalence : 0.4694
##          Balanced Accuracy : 0.7150
##
##          'Positive' Class : 1
##
```

Table 2.4 - Probit Model Confusion Matrix for Train Set

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction     0          1
##          0 7669 2152
##          1 3223 5526
##
##                    Accuracy : 0.7106
##                    95% CI : (0.704, 0.7171)
##          No Information Rate : 0.5865
##          P-Value [Acc > NIR] : < 2.2e-16
##
##                    Kappa : 0.4153
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.7197
##          Specificity : 0.7041
##          Pos Pred Value : 0.6316
##          Neg Pred Value : 0.7809
##          Prevalence : 0.4135
##          Detection Rate : 0.2976
##          Detection Prevalence : 0.4711
##          Balanced Accuracy : 0.7119
##
##          'Positive' Class : 1
##
```

Table 2.5 - Probit Model Confusion Matrix for Test Set

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction     0          1
```

```
##          0 25852  8985
##          1  6826 14050
##
##                Accuracy : 0.7162
##                  95% CI : (0.7124, 0.7199)
##     No Information Rate : 0.5865
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4067
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.6099
##             Specificity : 0.7911
##          Pos Pred Value : 0.6730
##          Neg Pred Value : 0.7421
##              Prevalence : 0.4135
##          Detection Rate : 0.2522
##    Detection Prevalence : 0.3747
##       Balanced Accuracy : 0.7005
##
##        'Positive' Class : 1
##
```

Table 2.6 - Elastic Net Model Confusion Matrix for Train Set

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0 8589 3035
##          1 2303 4643
##
##                Accuracy : 0.7125
##                  95% CI : (0.706, 0.719)
##     No Information Rate : 0.5865
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3989
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.6047
##             Specificity : 0.7886
##          Pos Pred Value : 0.6684
##          Neg Pred Value : 0.7389
##              Prevalence : 0.4135
##          Detection Rate : 0.2500
##    Detection Prevalence : 0.3740
```

```
##          Balanced Accuracy : 0.6966
##
##          'Positive' Class : 1
##
```

Table 2.7 - Elastic Net Model Confusion Matrix for Test Set

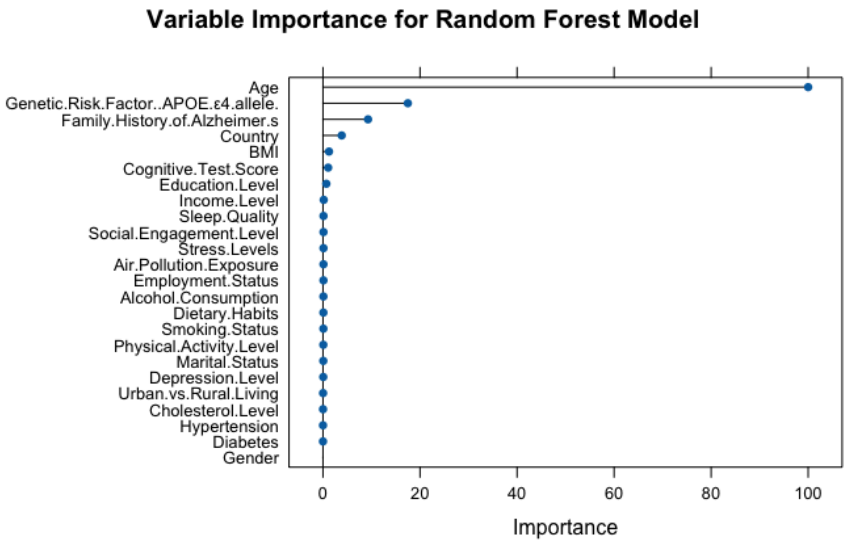

Table 2.8 - Random Forrest Feature Importance

```
## Confusion Matrix for Random Forest Model Test Set:

## Confusion Matrix and Statistics
 ##
##          Reference
 ## Prediction   X0   X1
 ##         X0 7619 1878
 ##         X1 3273 5800
 ##
##               Accuracy : 0.7226
##                 95% CI : (0.7161, 0.729)
 ##    No Information Rate : 0.5865
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                  Kappa : 0.443
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.7554
##             Specificity : 0.6995
##          Pos Pred Value : 0.6393
##          Neg Pred Value : 0.8023
##              Prevalence : 0.4135
##          Detection Rate : 0.3123
##    Detection Prevalence : 0.4886
##       Balanced Accuracy : 0.7275
##
##       'Positive' Class : X1
##
```

Table 2.9 - Random Forrest Confusion Matrix

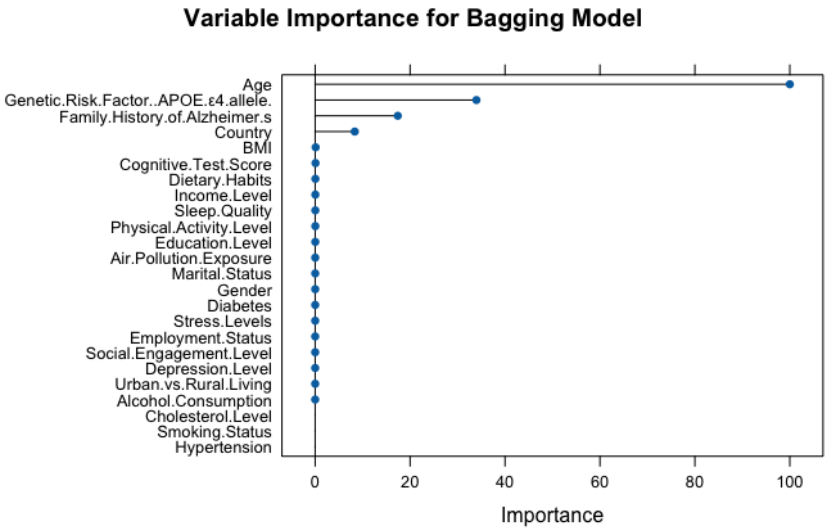

Table 3.0 - Bagging Feature Importance

```
## Confusion Matrix for Bagging Model Test Set:

## Confusion Matrix and Statistics
 ##
##          Reference
 ## Prediction   X0   X1
 ##         X0 7862 2097
 ##         X1 3030 5581
 ##
##               Accuracy : 0.7239
##                 95% CI : (0.7174, 0.7303)
 ##    No Information Rate : 0.5865
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                  Kappa : 0.4408
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
 ##            Sensitivity : 0.7269
##            Specificity : 0.7218
##         Pos Pred Value : 0.6481
##         Neg Pred Value : 0.7894
##             Prevalence : 0.4135
##         Detection Rate : 0.3005
##   Detection Prevalence : 0.4637
##      Balanced Accuracy : 0.7243
##
##       'Positive' Class : X1
##
```

Table 3.1 - Bagging Confusion Matrix

Section 3 – R Code

Note: The entirety of the code shown below can be found in the attached R Markdown file in addition to here.

```{r}
# Initializing libraries
library(caret)
library(glmnet)
library(dplyr)
library(MASS)
library(ggplot2)
library(tidyverse)
library(reshape2)

# Import the Dataset
data <- read.csv("alzheimers_prediction_dataset.csv", sep = ',')

# Summary of the data
colnames(data)


## Data Cleaning

levels <- c("Low", "Medium", "High")

l_m_h_factors <- c("Physical.Activity.Level", "Depression.Level", "Air.Pollution.Exposure", "Income.Level", "Social.Engagement.Level", "Stress.Levels") for (col in l_m_h_factors) { data[[col]] <- factor(data[[col]], levels = levels) }

data$Alcohol.Consumption <- factor(data$Alcohol.Consumption, levels = c("Never", "Occasionally", "Regularly")) data$Smoking.Status <- factor(data$Smoking.Status, levels = c("Never", "Former", "Current")) data$Dietary.Habits <- factor(data$Dietary.Habits, levels = c("Unhealthy", "Average", "Healthy"))

for (col in names(data)) { if (is.character(data[[col]])) { data[[col]] <- factor(data[[col]]) } }

#Create a new data set with all numeric values new_data <- data.frame()

factor_list <- c("Physical.Activity.Level", "Depression.Level", "Air.Pollution.Exposure", "Income.Level", "Social.Engagement.Level", "Stress.Levels", "Alcohol.Consumption", "Smoking.Status", "Dietary.Habits", "Country", "Gender", "Diabetes", "Hypertension", "Cholesterol.Level", "Family.History.of.Alzheimer.s", "Sleep.Quality", "Employment.Status", "Marital.Status", "Genetic.Risk.Factor..APOE.ε4.allele.", "Urban.vs.Rural.Living", "Alzheimer.s.Diagnosis")

#Convert the specified columns to numeric and combine them into a data frame new_data <- as.data.frame(lapply(factor_list, function(col) as.numeric(data[[col]])))

#Optionally, set the column names correctly names(new_data) <- factor_list

head(data)


levels <- c("Low", "Medium", "High")

l_m_h_factors <- c("Physical.Activity.Level", "Depression.Level", "Air.Pollution.Exposure", "Income.Level", "Social.Engagement.Level", "Stress.Levels") for (col in l_m_h_factors) { data[[col]] <- factor(data[[col]], levels = levels) }

data$Alcohol.Consumption <- factor(data$Alcohol.Consumption, levels = c("Never", "Occasionally", "Regularly")) data$Smoking.Status <- factor(data$Smoking.Status, levels = c("Never", "Former", "Current")) data$Dietary.Habits <- factor(data$Dietary.Habits, levels = c("Unhealthy", "Average", "Healthy"))
```

```
for (col in names(data)) { if (is.character(data[[col]])) { data[[col]] <- factor(data[[col]]) } }

#Create a new data set with all numeric values new_data <- data.frame()

factor_list <- c("Physical.Activity.Level", "Depression.Level", "Air.Pollution.Exposure", "Income.Level",
"Social.Engagement.Level", "Stress.Levels", "Alcohol.Consumption", "Smoking.Status", "Dietary.Habits", "Country", "Gender",
"Diabetes", "Hypertension", "Cholesterol.Level", "Family.History.of.Alzheimer.s", "Sleep.Quality", "Employment.Status",
"Marital.Status", "Genetic.Risk.Factor..APOE.ε4.allele.", "Urban.vs.Rural.Living", "Alzheimer.s.Diagnosis")

#Convert the specified columns to numeric and combine them into a data frame new_data <- as.data.frame(lapply(factor_list,
function(col) as.numeric(data[[col]])))

#Optionally, set the column names correctly names(new_data) <- factor_list

head(data)



## 1 – Exploratory Data Anslysis
#Extract numeric columns from 'data'

numeric_data <- data[, sapply(data, is.numeric)]

summary(data$Alzheimer.s.Diagnosis)

sum(is.na(data))


# Create univariate histograms with different transformations color-coded

ggplot(data, aes(x = data$Age)) + geom_histogram()

ggplot(data, aes(x = data$BMI)) + geom_histogram()

ggplot(data, aes(x = data$Education.Level)) + geom_histogram()

ggplot(data, aes(x = data$Cognitive.Test.Score)) + geom_histogram()


ggplot(data, aes(x = log(data$Age))) + geom_histogram(color="blue")

ggplot(data, aes(x = log(data$BMI))) + geom_histogram(color="blue")

ggplot(data, aes(x = log(data$Education.Level))) + geom_histogram(color="blue")

ggplot(data, aes(x = log(data$Cognitive.Test.Score))) + geom_histogram(color="blue")


ggplot(data, aes(x = sqrt(data$Age))) + geom_histogram(color="red")

ggplot(data, aes(x = sqrt(data$BMI))) + geom_histogram(color="red")

ggplot(data, aes(x = sqrt(data$Cognitive.Test.Score))) + geom_histogram(color="red")


ggplot(data, aes(x = 1/(data$Age))) + geom_histogram(color="yellow")
```

```
ggplot(data, aes(x = 1/(data$BMI))) + geom_histogram(color="yellow")

ggplot(data, aes(x = 1/(data$Cognitive.Test.Score))) + geom_histogram(color="yellow")


ggplot(data, aes(x = sqrt(1/(data$Age)))) + geom_histogram(color="orange")

ggplot(data, aes(x = sqrt(1/(data$BMI)))) + geom_histogram(color="orange")

ggplot(data, aes(x = sqrt(1/(data$Cognitive.Test.Score)))) + geom_histogram(color="orange")


ggplot(data, aes(x = log(1/(data$Age)))) + geom_histogram(color="green")

ggplot(data, aes(x = log(1/(data$BMI)))) + geom_histogram(color="green")

ggplot(data, aes(x = log(1/(data$Cognitive.Test.Score)))) + geom_histogram(color="green")


# View the means and medians

data %>%
	group_by_("Alzheimer.s.Diagnosis") %>%
	summarise(
		mean = mean(Age),
			median = median(Age),
			n = n()
		)

data %>%
	group_by_("Alzheimer.s.Diagnosis") %>%
	summarise(
		mean = mean(BMI),
		median = median(BMI),
		n = n()
	)

data %>%
	group_by_("Alzheimer.s.Diagnosis") %>%
	summarise(
		mean = mean(Education.Level),
		median = median(Education.Level),
		n = n()
	)

data %>%
	group_by_("Alzheimer.s.Diagnosis") %>%
	summarise(
		mean = mean(Cognitive.Test.Score),
		median = median(Cognitive.Test.Score),
		n = n()
	)


# Create bar charts for factor variables
```

```
# Select only categorical variables
cat.data <- data[, sapply(data, is.factor)]


# Loop through categorical columns
for (i in colnames(cat.data)) {
  plot <- ggplot(data, aes(x = .data[[i]], fill = factor(Alzheimer.s.Diagnosis))) + # Select Age as what to compare each to
    geom_bar(position = "fill") +
    labs(x = i, y = "percent") +
    theme(axis.text.x = element_text(angle = 90, hjust = 1))


  print(plot)  # Print each plot
```

```
# Look at the mean of Alzheimer’s Diagnosis by different levels

test1 <- data[, sapply(data, is.factor)]

# Convert Alzheimer.s.Diagnosis to numeric

test1$Alzheimer.s.Diagnosis <- as.numeric(test1$Alzheimer.s.Diagnosis) - 1

 # Perform the group_by and summarise operations

x <- test1 %>%

  group_by(Country) %>%

  summarise(mean = mean(Alzheimer.s.Diagnosis, na.rm = TRUE), n = n())


print(x)
```

```
# See what would happen if we combined multiple countries into continents., or what would happen if we grouped all countries with a mean of >0.42 together.

levels(data$Country)

table(data$Country)

test2 <- data[, sapply(data, is.factor)]

# Group by continent:

levels(test2$Country)[levels(test2$Country) == "Brazil"] <- "South America"

levels(test2$Country)[levels(test2$Country) == "Argentina"] <- "South America"

levels(test2$Country)[levels(test2$Country) == "Canada"] <- "North America"

levels(test2$Country)[levels(test2$Country) == "USA"] <- "North America"

levels(test2$Country)[levels(test2$Country) == "Mexico"] <- "North America"

levels(test2$Country)[levels(test2$Country) == "China"] <- "Asia"

levels(test2$Country)[levels(test2$Country) == "India"] <- "Asia"

levels(test2$Country)[levels(test2$Country) == "Japan"] <- "Asia"

levels(test2$Country)[levels(test2$Country) == "Saudi Arabia"] <- "Asia"

levels(test2$Country)[levels(test2$Country) == "South Korea"] <- "Asia"

levels(test2$Country)[levels(test2$Country) == "France"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "Germany"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "Sweden"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "Russia"] <- "Europe"


levels(test2$Country)[levels(test2$Country) == "Spain"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "UK"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "Italy"] <- "Europe"

levels(test2$Country)[levels(test2$Country) == "Norway"] <- "Europe"


# Convert Alzheimer.s.Diagnosis to numeric

test2$Alzheimer.s.Diagnosis <- as.numeric(test1$Alzheimer.s.Diagnosis)


# Perform the group_by and summarise operations, then print each
```

```
x <- test2 %>%
  group_by(Country) %>%
  summarise(mean = mean(Alzheimer.s.Diagnosis, na.rm = TRUE), n = n())

print(x)

# Variable to test for mean >0.42
test3 <- test1

levels(test3$Country)[levels(test3$Country) == "Russia"] <- "Russia-India-Brazil-Mexico-SouthAfrica"
levels(test3$Country)[levels(test3$Country) == "India"] <- "Russia-India-Brazil-Mexico-SouthAfrica"
levels(test3$Country)[levels(test3$Country) == "Brazil"] <- "Russia-India-Brazil-Mexico-SouthAfrica"
levels(test3$Country)[levels(test3$Country) == "Mexico"] <- "Russia-India-Brazil-Mexico-SouthAfrica"
levels(test3$Country)[levels(test3$Country) == "South Africa"] <- "Russia-India-Brazil-Mexico-SouthAfrica"
levels(test3$Country)[!levels(test3$Country) == "Russia-India-Brazil-Mexico-SouthAfrica"] <- "Other"

# Perform the group_by and summarise operations, then print each
x <- test3 %>%
  group_by(Country) %>%
  summarise(mean = mean(Alzheimer.s.Diagnosis, na.rm = TRUE), n = n())
  print(x)
```

```
# Find the correlation matrix for the numerical predictors

cor.matrix.numerical <- cor(data[, c(2, 4, 5, 13)])

round(cor.matrix.numerical, digits = 4)


# Find the correlation matrix and print the figure for the entire dataset
cor.matrix.all <- cor(new_data)

round(cor.matrix.all, digits = 2)

melted.cor.matrix <- melt(round(cor.matrix.all, digits = 4))

ggplot(data = melted.cor.matrix, aes(x=Var1, y=Var2, fill=value)) + geom_raster()


test4 <- data


levels(test4$Country)[levels(test4$Country) == "Russia"] <- "Russia-India-Brazil-Mexico-SouthAfrica"

levels(test4$Country)[levels(test4$Country) == "India"] <- "Russia-India-Brazil-Mexico-SouthAfrica"

levels(test4$Country)[levels(test4$Country) == "Brazil"] <- "Russia-India-Brazil-Mexico-SouthAfrica"

levels(test4$Country)[levels(test4$Country) == "Mexico"] <- "Russia-India-Brazil-Mexico-SouthAfrica"

levels(test4$Country)[levels(test4$Country) == "South Africa"] <- "Russia-India-Brazil-Mexico-SouthAfrica"


levels(test4$Country)[!levels(test4$Country) == "Russia-India-Brazil-Mexico-SouthAfrica"] <- "Other"


# Loop through categorical columns

for (i in colnames(test4)) {

  plot <- ggplot(test4, aes(x = .data[[i]], fill = factor(Age))) + # Select Age as what to compare each to

    geom_bar(position = "fill") +

    labs(x = i, y = "percent") +

    theme(axis.text.x = element_text(angle = 90, hjust = 1))


  print(plot)  # Print each plot

# Print plots to see correlation between genetic risk factor/age, and other variables before and after country condesing was applied
```

```
  plot <- ggplot(data, aes(x = .data[["Country"]], fill = factor(Genetic.Risk.Factor..APOE.ε4.allele.))) +
  geom_bar(position = "fill") +
  labs(x = "Country", y = "percent") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

  print(plot)

  plot <- ggplot(data, aes(x = .data[["Country"]], fill = factor(Age))) + # Select Age as what to compare each to
  geom_bar(position = "fill") +
  labs(x = "Country", y = "percent") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

  print(plot)
}


# Loop through categorical columns
for (i in colnames(test4)) {
  plot <- ggplot(test4, aes(x = .data[[i]], fill = factor(Genetic.Risk.Factor..APOE.ε4.allele.))) +
    geom_bar(position = "fill") +
    labs(x = i, y = "percent") +
    theme(axis.text.x = element_text(angle = 90, hjust = 1))

  print(plot)  # Print each plot
}
 data <- test4 #Set the new data we are working with to the version with condensed countries
```

```
# Testing interaction between Diabetes/Diet, Physical Activity Level/BMI, and Diet/BMI
# Ensure 'Alzheimer.s.Diagnosis' and 'Diabetes' are factors
data$Alzheimer.s.Diagnosis <- as.factor(data$Alzheimer.s.Diagnosis)
data$Diabetes <- as.factor(data$Diabetes)

# Create the plot
ggplot(data, aes(x = Diabetes, fill = Alzheimer.s.Diagnosis)) +
  geom_bar(position = "fill") +
  facet_wrap(~ Dietary.Habits) +
  labs(y = "Proportion", x = "Diabetes") +
  theme_minimal()
ggplot(data, aes( x = Alzheimer.s.Diagnosis, y = BMI)) + geom_boxplot() + facet_wrap(~ Dietary.Habits)
ggplot(data, aes( x = Alzheimer.s.Diagnosis, y = BMI)) + geom_boxplot() + facet_wrap(~ Physical.Activity.Level)

## 2 – Model Fitting

# Convert the specified columns to numeric and combine them into a data frame
new_data <- as.data.frame(lapply(factor_list, function(col) as.numeric(data[[col]])))
# Optionally, set the column names correctly
names(new_data) <- factor_list
# Extract numeric columns from 'data'
numeric_data <- data[, sapply(data, is.numeric)]
# Assuming new_data is already defined (even as an empty data frame with the same number of rows)
new_data <- cbind(new_data, numeric_data)
#Transform the Alzheimer Diagnosis from 1 and 2 to 0 and 1
new_data$Alzheimer.s.Diagnosis <- as.numeric(as.character(new_data$Alzheimer.s.Diagnosis)) - 1
head(new_data)
```

```
# Partition the data so that we can have the train and test set to evaluate the models used

set.seed(123)

# Create a stratified partition for a 70/30 split

train_index <- createDataPartition(data$Alzheimer.s.Diagnosis, p = 0.75, list = FALSE)

# Split the data into training and test sets

train_data_log <- data[train_index, ] test_data_log <- data[-train_index, ]

train_data_glm <- new_data[train_index, ] test_data_glm <- new_data[-train_index, ]

#Transform target column for the logistic data train_data_log$Alzheimer.s.Diagnosis <-
as.numeric(train_data_log$Alzheimer.s.Diagnosis) - 1 test_data_log$Alzheimer.s.Diagnosis <-
as.numeric(test_data_log$Alzheimer.s.Diagnosis) - 1

head(train_data_log)



# Conduct logistic regression

factor_levels <- sapply(train_data_log, function(x) if (is.factor(x)) length(unique(x)) else NA)

log_reg <- glm(Alzheimer.s.Diagnosis ~ . , data = train_data_log, family = binomial)

log_reg.offset <- glm(Alzheimer.s.Diagnosis ~ . - Age, data = train_data_log, offset = log(Age), family = binomial)



# Convert Alzheimer from Yes or No to binary numerical

data$Alzheimer.s.Diagnosis <- as.numeric(data$Alzheimer.s.Diagnosis == "Yes")

probit_reg <- glm(Alzheimer.s.Diagnosis ~ ., data = train_data_log, family = binomial(link = "probit"))

AIC(log_reg) AIC(log_reg.offset) AIC(probit_reg)
```

```
# Create a confusion matrix for the model
cutoff <- 0.4141762

pred.train <- predict(probit_reg, type = "response") class.train <- ifelse(pred.train > cutoff, 1, 0)

confusionMatrix(factor(class.train), factor(train_data_log$Alzheimer.s.Diagnosis), positive = "1")

pred.test <- predict(probit_reg, newdata = test_data_log, type = "response") class.test <- ifelse(pred.test > cutoff, 1, 0)

confusionMatrix(factor(class.test), factor(test_data_log$Alzheimer.s.Diagnosis), positive ="1")
```

```
# Binarization
binarizer <- dummyVars(~ Country + Physical.Activity.Level + Smoking.Status + Alcohol.Consumption + Depression.Level +
Sleep.Quality + Dietary.Habits + Air.Pollution.Exposure + Employment.Status + Marital.Status + Social.Engagement.Level +
Income.Level + Stress.Levels,

                    data = data,

                    fullRank = T)

binarized.vars <- data.frame(predict(binarizer, newdata = data))

head(binarized.vars)



data.bin <- cbind(data, binarized.vars)
data.bin$Country <- NULL
data.bin$Physical.Activity.Level <- NULL
data.bin$Smoking.Status <- NULL
data.bin$Alcohol.Consumption <- NULLdata.bin$Depression.Level <- NULL
data.bin$Sleep.Quality <- NULL
data.bin$Dietary.Habits <- NULL
data.bin$Air.Pollution.Exposure <- NULL
data.bin$Employment.Status <- NULL
data.bin$Marital.Status <- NULL
data.bin$Social.Engagement.Level <- NULL
data.bin$Income.Level <- NULL
data.bin$Stress.Levels <- NULL
data.train.bin <- data.bin[train_index, ]
data.test.bin <- data.bin[-train_index, ]

head(data.train.bin)


# Fit the new Probit model with binarization

probit.full <- glm(Alzheimer.s.Diagnosis ~ .,

                   data = data.train.bin,

                   family = binomial("probit"))

probit.null <- glm(Alzheimer.s.Diagnosis ~ 1,

                   data = data.train.bin,

                   family = binomial("probit"))

probit.reduced <- stepAIC(probit.null,

                      direction = "forward",

                      scope = list(upper = probit.full, lower = probit.null),

                      trace = 0)
```

```
summary(probit.reduced) # Summary of reduced model

cutoff <- 0.4141762
```

```
# Confusion Matrix for Reduced Model
pred.train.reduced <- predict(probit.reduced, type = "response")
class.train.reduced <- ifelse(pred.train.reduced > cutoff, 1, 0)

confusionMatrix(factor(class.train.reduced), factor(data.train.bin$Alzheimer.s.Diagnosis), positive = "1")

pred.test.reduced <- predict(probit.reduced, newdata = data.test.bin, type = "response")
class.test.reduced <- ifelse(pred.test.reduced > cutoff, 1, 0)

confusionMatrix(factor(class.test.reduced), factor(data.test.bin$Alzheimer.s.Diagnosis), positive ="1")
```

```
alpha_val <- 0.75

#Assign input values for glmnet compatibility x_train <- model.matrix(Alzheimer.s.Diagnosis ~ . - 1, data = train_data_glm)
x_test <- model.matrix(Alzheimer.s.Diagnosis ~ . - 1, data = test_data_glm)

y_train <- train_data_glm$Alzheimer.s.Diagnosis y_test <- test_data_glm$Alzheimer.s.Diagnosis

#Transform to numeric to plot lambda y_train_numeric <- as.numeric(as.character(y_train)) y_test_numeric <-
as.numeric(as.character(y_test))

# PERFORM CROSS-VALIDATION TO SEARCH FOR THE OPTIMAL LAMBDA

lambda_Search <- cv.glmnet(x_train, y_train_numeric, alpha = alpha_val, nfolds = 10, family = "binomial")

# PLOT THE CROSS-VALIDATION ERROR VERSUS LAMBDA.

lambda_Search$lambda.min lambda_Search$lambda.1se plot(lambda_Search)



# Fit the elastic net logisitc regression model

log_elastic_net <- glmnet(x_train, y_train, family = "binomial", lambda =
lambda_Search$lambda.min, alpha = alpha_val)
print(log_elastic_net)
coef_opt <- coef(log_elastic_net, s = lambda_Search$lambda.min)
print(coef_opt)
```

Train and Test Accuracy of the Logistic Elastic Net Model

pred_prob_train <- predict(log_elastic_net, newx = x_train, s = lambda_Search$lambda.min, type = "response")

Convert predicted probabilities to class labels using a threshold of 0.5:

pred_class_train <- ifelse(pred_prob_train >= 0.5, 1, 0)

Create confusion matrix for the training set

confusion_matrix_train <- confusionMatrix(factor(pred_class_train), factor(y_train), positive = '1') cat("Confusion Matrix for Training Set:\n") print(confusion_matrix_train)

Calculate accuracy (percent correct) for the training set:

accuracy_train <- mean(pred_class_train == y_train) cat("Training Accuracy:", accuracy_train, "\n")

pred_prob_test <- predict(log_elastic_net, newx = x_test, s = lambda_Search$lambda.min, type = "response") pred_class_test <- ifelse(pred_prob_test >= 0.5, 1, 0)

Create confusion matrix for the test set

confusion_matrix_test <- confusionMatrix(factor(pred_class_test), factor(y_test), positive = '1') cat("Confusion Matrix for Test Set:\n") print(confusion_matrix_test)

Calculate accuracy (percent correct) for the training set:

accuracy_train <- mean(pred_class_test == y_test) cat("Training Accuracy:", accuracy_train, "\n")

Train & Test Accuracy of the Logistic Regression Model

Predict on the training set

train_pred_prob <- predict(log_reg, newdata = train_data_log, type = "response") train_pred_class <- ifelse(train_pred_prob >= 0.5, 1, 0)

Extract the actual values (ensure they are numeric 0/1)

train_actual <- as.numeric(as.character(train_data_log$Alzheimer.s.Diagnosis))

Create confusion matrix for the test set

confusion_matrix_test <- confusionMatrix(factor(train_pred_class), factor(train_actual), positive = '1') cat("Confusion Matrix for Test Set:\n") print(confusion_matrix_test)

```
# Calculate accuracy (percent correct) for the training set:

accuracy_train <- mean(train_pred_class == train_actual) cat("Training Accuracy:", accuracy_train, "\n")

# Predict on the test set

test_pred_prob <- predict(log_reg, newdata = test_data_log, type = "response") test_pred_class <- ifelse(test_pred_prob >= 0.5, 1, 0)

# Extract the actual values from the test set

test_actual <- as.numeric(as.character(test_data_log$Alzheimer.s.Diagnosis))

# Create confusion matrix for the test set

confusion_matrix_test <- confusionMatrix(factor(test_pred_class), factor(test_actual), positive = '1') cat("Confusion Matrix for Test Set:\n") print(confusion_matrix_test)

# Calculate accuracy (percent correct) for the training set:

accuracy_train <- mean(test_pred_class == test_actual) cat("Training Accuracy:", accuracy_train, "\n")

# Predict on the Offset data

train_pred_prob <- predict(log_reg.offset, newdata = train_data_log, type = "response") train_pred_class <- ifelse(train_pred_prob >= 0.5, 1, 0)

# Extract the actual values (ensure they are numeric 0/1)

train_actual <- as.numeric(as.character(train_data_log$Alzheimer.s.Diagnosis))

# Create confusion matrix for the test set

confusion_matrix_test <- confusionMatrix(factor(train_pred_class), factor(train_actual), positive = '1') cat("Confusion Matrix for Test Set:\n") print(confusion_matrix_test)

# Calculate accuracy (percent correct) for the training set:

accuracy_train <- mean(train_pred_class == train_actual) cat("Training Accuracy:", accuracy_train, "\n")

# Predict on the test set

test_pred_prob <- predict(log_reg.offset, newdata = test_data_log, type = "response") test_pred_class <- ifelse(test_pred_prob >= 0.5, 1, 0)

# Extract the actual values from the test set

test_actual <- as.numeric(as.character(test_data_log$Alzheimer.s.Diagnosis))
```

```
# Create confusion matrix for the test set
confusion_matrix_test <- confusionMatrix(factor(test_pred_class), factor(test_actual), positive = '1')
cat("Confusion Matrix for Test Set:\n")
print(confusion_matrix_test)
#Calculate accuracy (percent correct) for the training set:
accuracy_train <- mean(test_pred_class == test_actual)
cat("Training Accuracy:", accuracy_train, "\n")
```